

# CPSC 340: Machine Learning and Data Mining

More Regularization

# Admin

- No class Monday (Oct 13, Thanksgiving)
- A3 due Wednesday (Oct 15)

# Last Time: L2-Regularization

- We discussed **regularization**:

- Adding a **continuous penalty on the model complexity**:

$$f(w) = \frac{1}{2} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2$$

- Best parameter  $\lambda$  almost **always leads to improved test error**.

- L2-regularized least squares is also known as “**ridge regression**”.
- Can be **solved as a linear system** like least squares.

- Numerous other benefits:

- Solution is unique, less sensitive to data, gradient descent converges faster.

# Regularizing the y-Intercept?

- Should we **regularize the y-intercept**?
- No! Why encourage it to be closer to zero? (It could be anywhere.)
  - You should be allowed to shift function up/down globally.
- Yes! It makes the solution unique and it easier to compute 'w'.
- Compromise 1: regularize bias but center the  $y_i$  variables.
- Compromise 2: regularize bias by a **smaller amount**:

$$f(w, w_0) = \frac{1}{2} \sum_{i=1}^n (w^T x_i + w_0 - y_i)^2 + \frac{\lambda}{2} \|w\|^2 + \frac{\lambda_0}{2} w_0^2$$



# Parametric vs. Non-Parametric Transforms

- We've been using linear models with **polynomial bases**:

$$\hat{y}_i = w_0 \boxed{1} + w_1 \boxed{x_{i1}} + w_2 \boxed{(x_{i1})^2} + w_3 \boxed{(x_{i1})^3} + w_4 \boxed{(x_{i1})^4}$$

- But polynomials are not the only **possible bases**:
  - Exponentials, logarithms, trigonometric functions, etc.
  - The **right basis will vastly improve performance**.
  - If we use the wrong basis, our accuracy is limited even with lots of data.
  - But the **right basis may not be obvious**.

# Parametric vs. Non-Parametric Transforms

- We've been using linear models with **polynomial bases**:

$$\hat{y}_i = w_0 \boxed{1} + w_1 \boxed{x_{i1}} + w_2 \boxed{(x_{i1})^2} + w_3 \boxed{(x_{i1})^3} + w_4 \boxed{(x_{i1})^4}$$

- Alternative is **non-parametric** bases:
  - Size of basis (number of features) **grows with 'n'**.
  - Model gets more complicated as you get more data.
  - Can **model complicated functions** where you don't know the right basis.
    - With enough data.
  - Classic example is "**Gaussian RBFs**" ("Gaussian" == "normal distribution").

# Gaussian RBFs: A Sum of "Bumps"

Radial Base  
Function

$$\hat{y}_i = w_0 \boxed{\text{flat line}} + w_1 \boxed{\text{diagonal line}} + w_2 \boxed{\text{parabola}} + w_3 \boxed{\text{S-curve}} + w_4 \boxed{\text{U-shape}}$$

Polynomial basis represents function as sum of global polynomials.

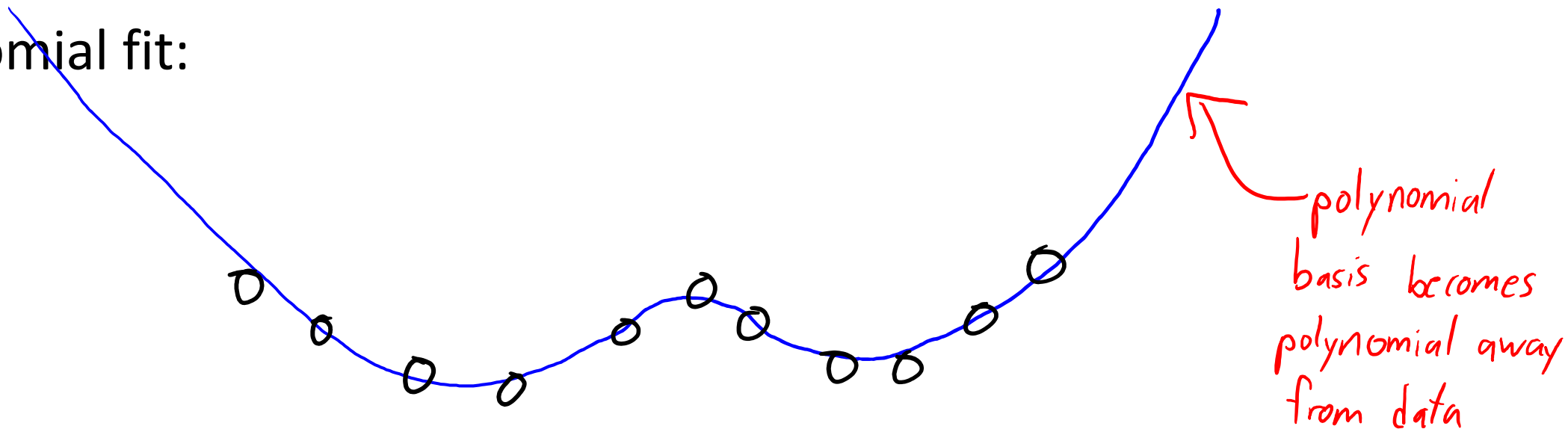
$$\hat{y}_i = w_0 \boxed{\text{bump 1}} + w_1 \boxed{\text{bump 2}} + w_2 \boxed{\text{bump 3}} + w_3 \boxed{\text{bump 4}} + w_4 \boxed{\text{bump 5}}$$

Gaussian RBFs represent function as sum of local "bumps"

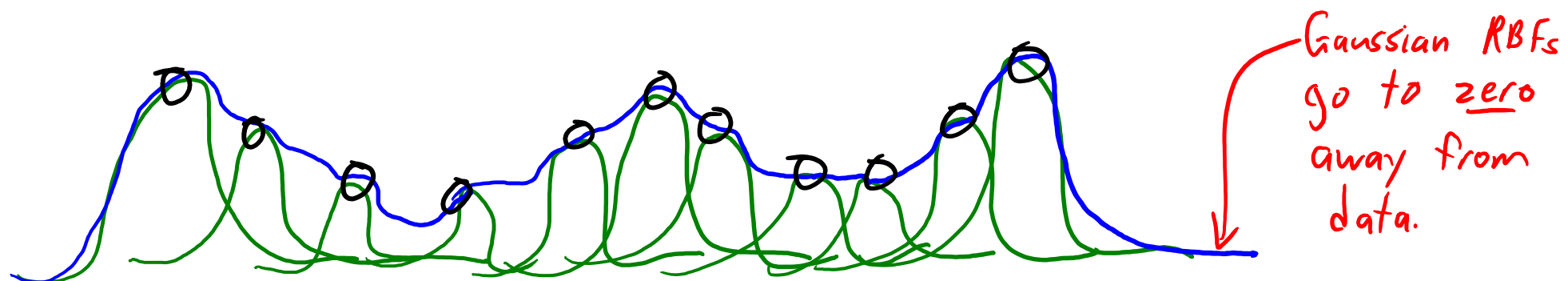
- Gaussian RBFs are **universal approximators** (compact subsets of  $\mathbb{R}^d$ )
  - Enough bumps can **approximate any continuous function** to arbitrary precision.
  - **Achieve optimal test error** as 'n' goes to infinity.

# Gaussian RBFs: A Sum of “Bumps”

- Polynomial fit:

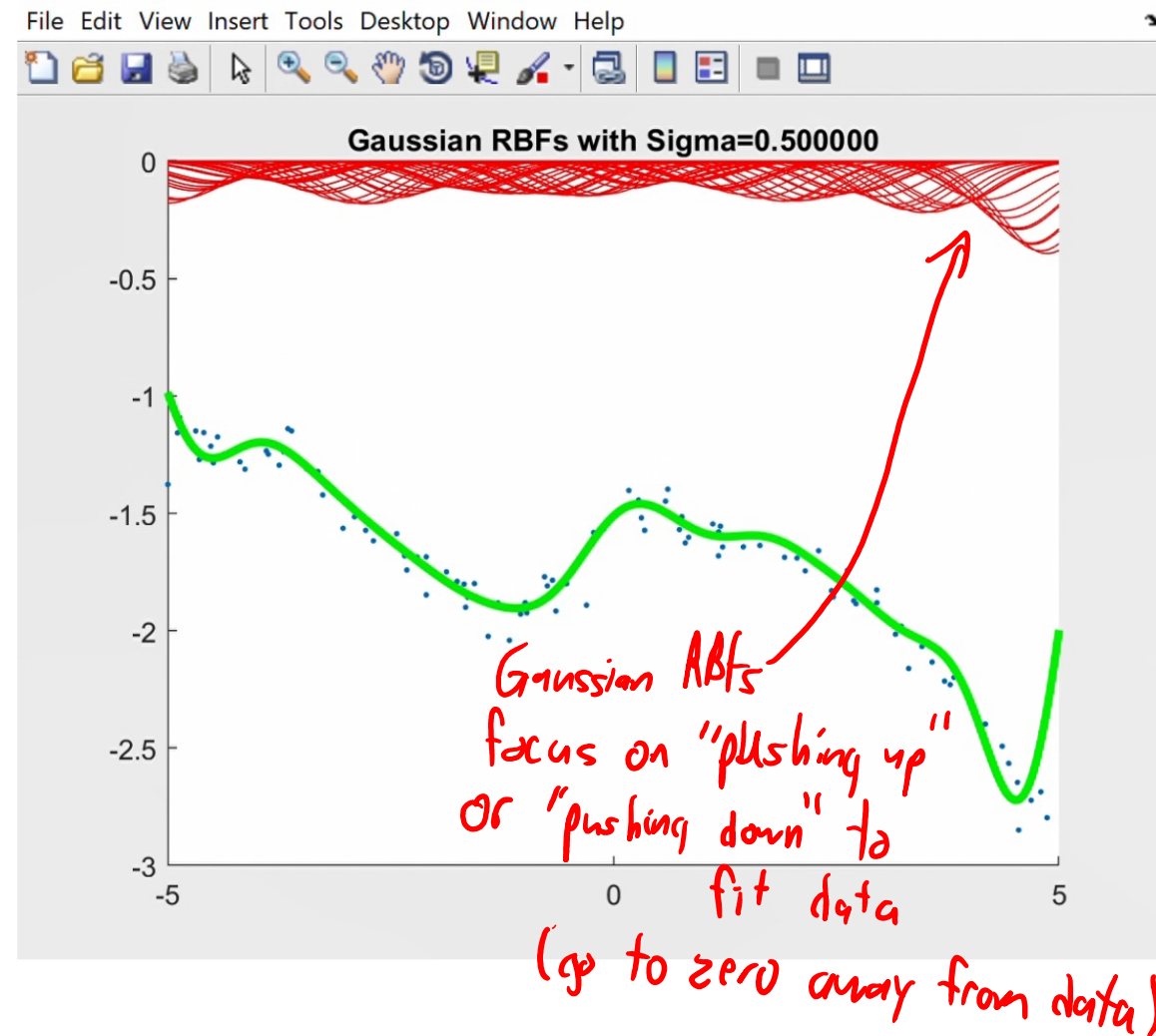
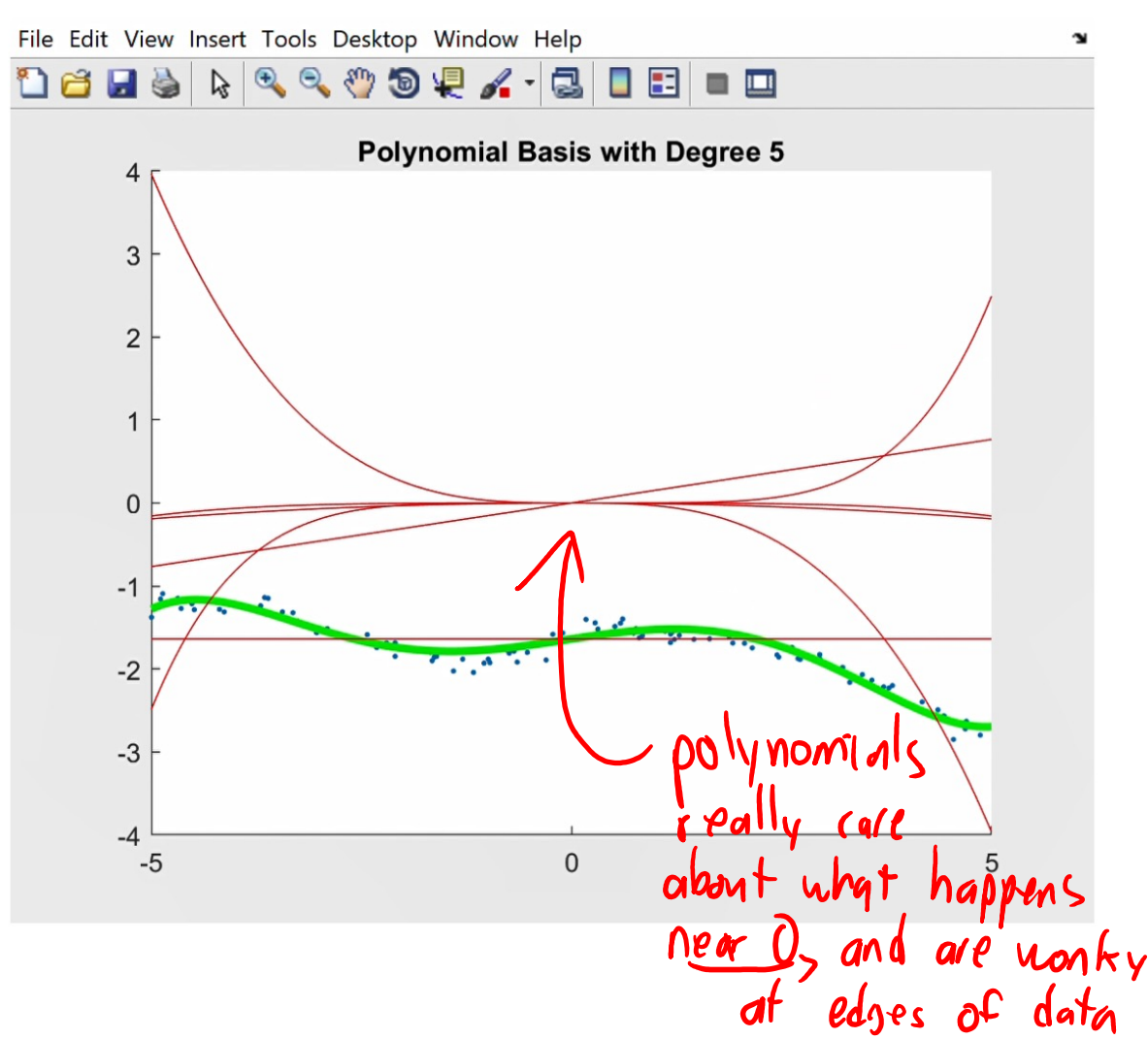


- Constructing a function from bumps (“smooth histogram”):



# Gaussian RBFs: A Sum of “Bumps”

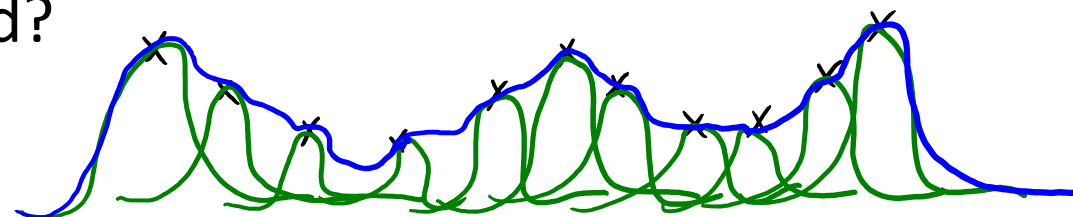
- Red is weight\*feature, green is prediction (sum of red lines):



# Gaussian RBF Parameters

- Some obvious questions:

1. How many bumps should we use?
2. Where should the bumps be centered?
3. How high should the bumps go?
4. How wide should the bumps be?



- The usual answers:

1. We use 'n' bumps (non-parametric basis).
2. Each bump is centered on one training example  $x_i$ .
3. Fitting regression weights 'w' gives us the heights (and signs).
4. The width is a hyper-parameter (narrow bumps == complicated model).

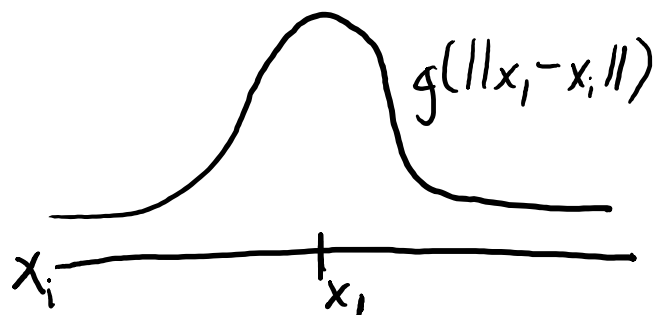
# Gaussian RBFs: Formal Details

- What is a **radial basis functions** (RBFs)?
  - A set of non-parametric bases that **depend on distances to training points**.

Replace  $x_i = (\underbrace{x_{i1}, x_{i2}, \dots, x_{id}}_{\text{'d' features}})$  with  $z_i = (\underbrace{g(\|x_i - x_1\|), g(\|x_i - x_2\|), \dots, g(\|x_i - x_n\|)}_{\text{'n' features}})$

*Handwritten notes:*  
 - Above  $g(\|x_i - x_1\|)$ : distance of Feature vector to example 1  
 - Above  $g(\|x_i - x_2\|)$ : distance to example 2

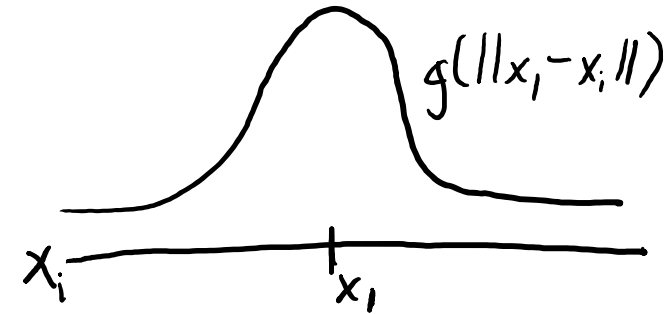
- Have 'n' features, with **feature 'i' depending on distance to example 'i'**.
  - Typically the **feature will decrease as the distance increases**:



# Gaussian RBFs: Formal Details

- What is a **radial basis functions** (RBFs)?
  - Most common choice of 'g' is **Gaussian RBF**:

$$g(\epsilon) = \exp\left(-\frac{\epsilon^2}{2\sigma^2}\right)$$



- **Variance  $\sigma^2$**  is a hyper-parameter controlling “width”.
  - This affects fundamental trade-off (set it using a validation set).

– Why don't we have  $\sqrt{2\pi\sigma}$  in the above formula?

- If you do not regularize it does not matter:
  - If 'v' is least squares solution with features  $z_i$ , then  $(\sqrt{2\pi\sigma})v$  is solution with features  $(1/\sqrt{2\pi\sigma})z_i$ .
  - So you **get the same predictions** (least squares is invariant to scaling of features).
- If you regularize it “sort of” matters:
  - It changes the effect of a fixed  $\lambda$ .
  - But the regularization path is the same, so if you search for the best  $\lambda$  you **get same predictions**.



Non-Parametric model is a feature transform.

## Gaussian RBFs: Formal Details

- What is a **radial basis functions** (RBFs)?
  - The training and testing matrices when using RBFs:

Replace  $X = \left[ \begin{array}{c} \text{ } \end{array} \right]_d^n$  by  $Z = \left[ \begin{array}{c} g(\|x_1 - x_1\|) \quad g(\|x_1 - x_2\|) \quad \dots \quad g(\|x_1 - x_n\|) \\ g(\|x_2 - x_1\|) \quad g(\|x_2 - x_2\|) \quad \dots \quad g(\|x_2 - x_n\|) \\ \vdots \quad \vdots \quad \ddots \quad \vdots \\ g(\|x_n - x_1\|) \quad g(\|x_n - x_2\|) \quad \dots \quad g(\|x_n - x_n\|) \end{array} \right]_n^n$

*Handwritten notes:* distance of 1st point to 1st point,  $\exp(-\dots)$

To make predictions on  $\tilde{X} = \left[ \begin{array}{c} \text{ } \end{array} \right]_d^t$  use  $\tilde{Z} = \left[ \begin{array}{c} g(\|\tilde{x}_i - x_j\|) \end{array} \right]_n^t$

*Handwritten notes:*

- $g(\|x - x_i\|) = e^{-\frac{\|x - x_i\|^2}{\sigma^2}}$
- features are  $n$
- Number of "features" is number of training examples

# Gaussian RBFs: Pseudo-Code

Constructing Gaussian RBFs given data 'X' and hyper-parameter  $\sigma$ :

```
Z = zeros(n, n)
```

```
for i1 in 1:n
```

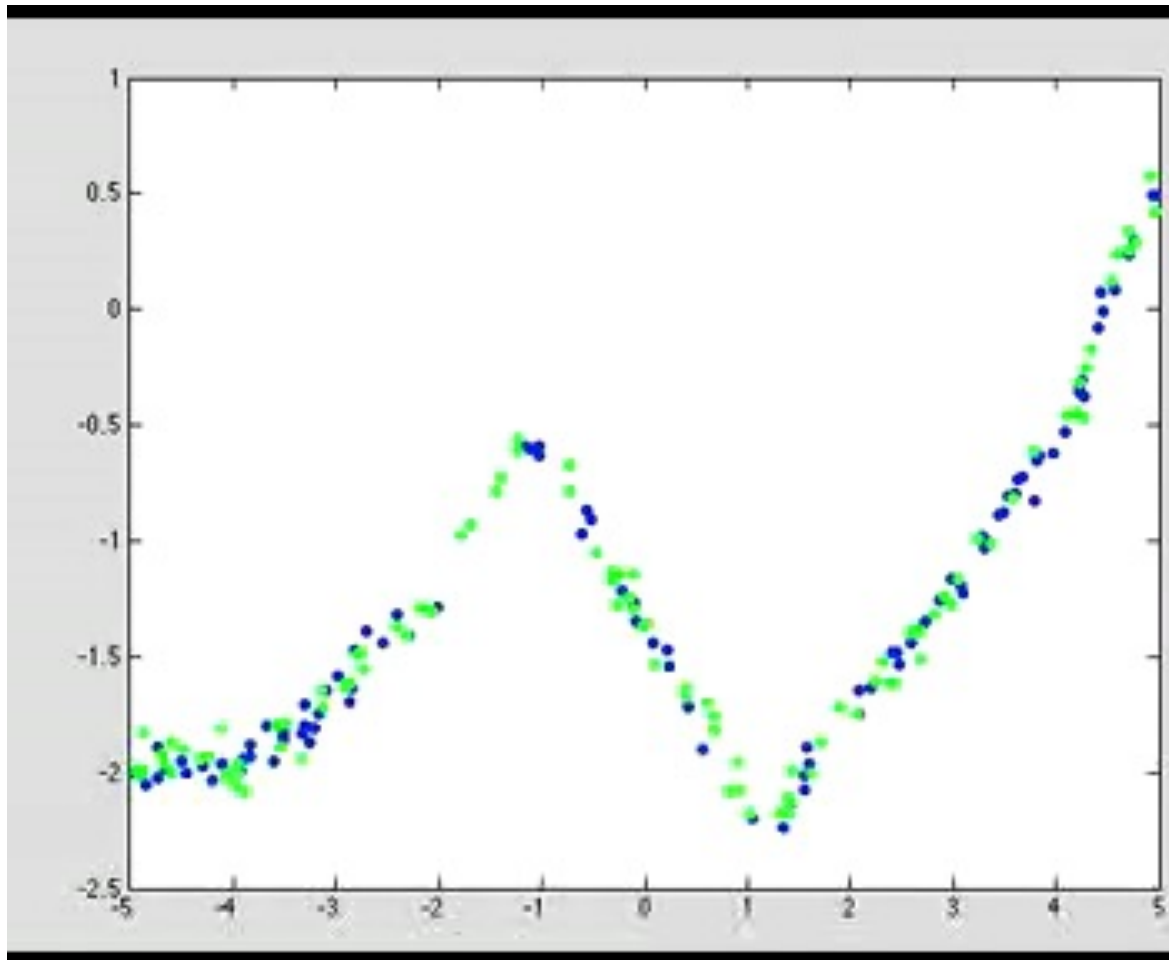
```
    for i2 in 1:n
```

```
        Z[i1, i2] = exp(-norm(X[i1, :] - X[i2, :])2 / (2 $\sigma^2$ ))
```

With test data  $\tilde{X}$ : form  $\tilde{Z}$  based on distances to training examples.

# Non-Parametric Basis: RBFs

- Least squares with Gaussian RBFs for different  $\sigma$  values:



Could add bias and linear basis:

$$Z = \begin{bmatrix} 1 & x_1 & g(\|x_1 - x_1\|) & \dots & g(\|x_1 - x_n\|) \\ 1 & x_2 & & & \\ 1 & x_3 & & & \\ \vdots & \vdots & & & \\ 1 & x_n & g(\|x_1 - x_n\|) & \dots & g(\|x_n - x_n\|) \end{bmatrix}$$

$\underbrace{\quad}_1 \quad \underbrace{\quad}_d \quad \underbrace{\quad}_n$

This reverts to linear regression instead of 0 away from data.

# RBFs and Regularization

- Gaussian Radial basis functions (RBFs) predictions:

$$\begin{aligned}\hat{y}_i &= w_1 \exp\left(-\frac{\|x_i - x_1\|^2}{2\sigma^2}\right) + w_2 \exp\left(-\frac{\|x_i - x_2\|^2}{2\sigma^2}\right) + \dots + w_n \exp\left(-\frac{\|x_i - x_n\|^2}{2\sigma^2}\right) \\ &= \sum_{j=1}^n w_j \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)\end{aligned}$$

- Flexible bases that can model any continuous function.
  - But with 'n' data points RBFs have 'n' basis functions.
- How do we avoid overfitting with this huge number of features?
    - We regularize 'w' and use validation error to choose  $\sigma$  and  $\lambda$ .

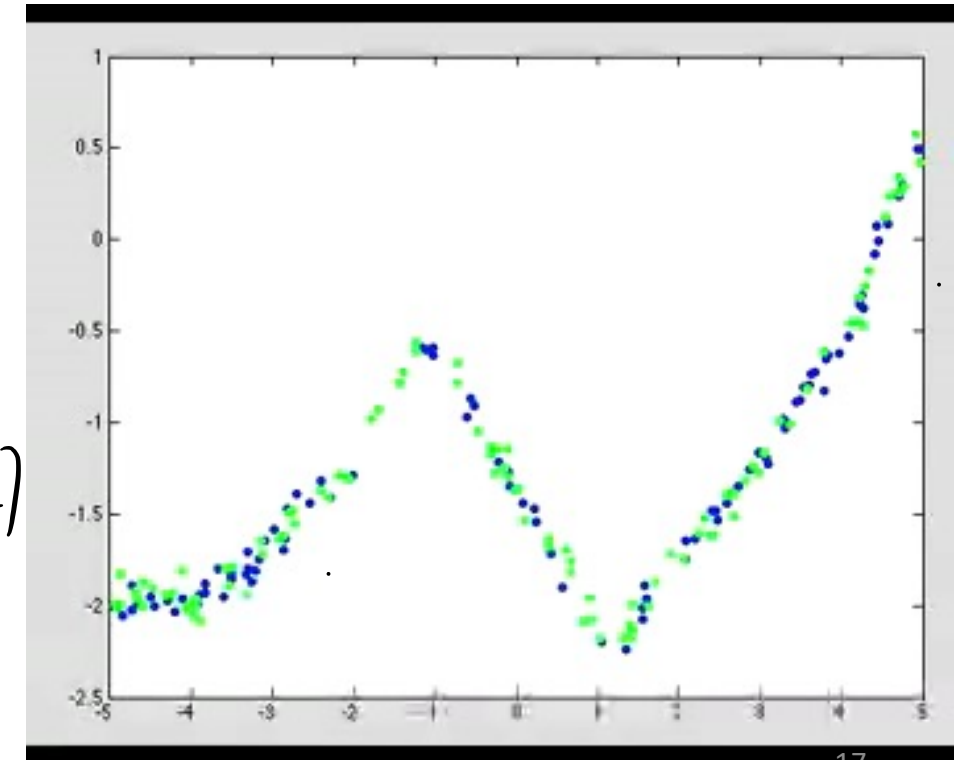
# RBFs, Regularization, and Validation

- A model that is hard to beat:
  - RBF basis with L2-regularization and cross-validation to choose  $\sigma$  and  $\lambda$ .
  - Flexible non-parametric basis, magic of regularization, and tuning for test error.

Non-Parametric  
model  $\rightarrow$   
Better

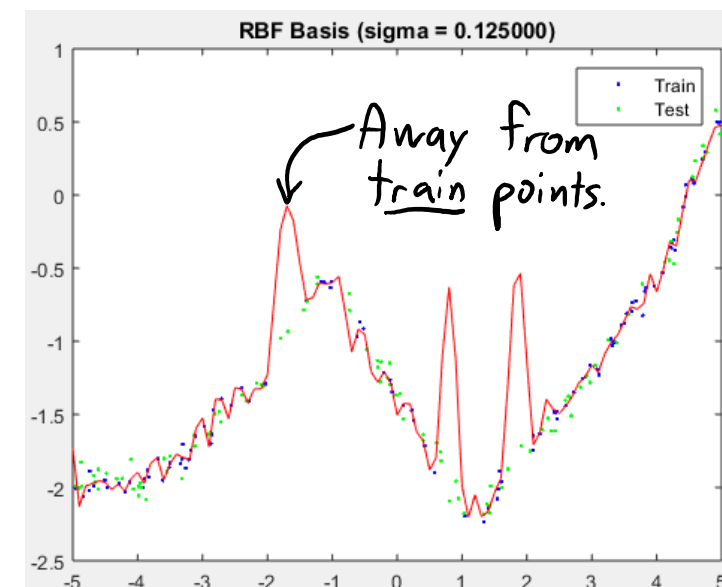
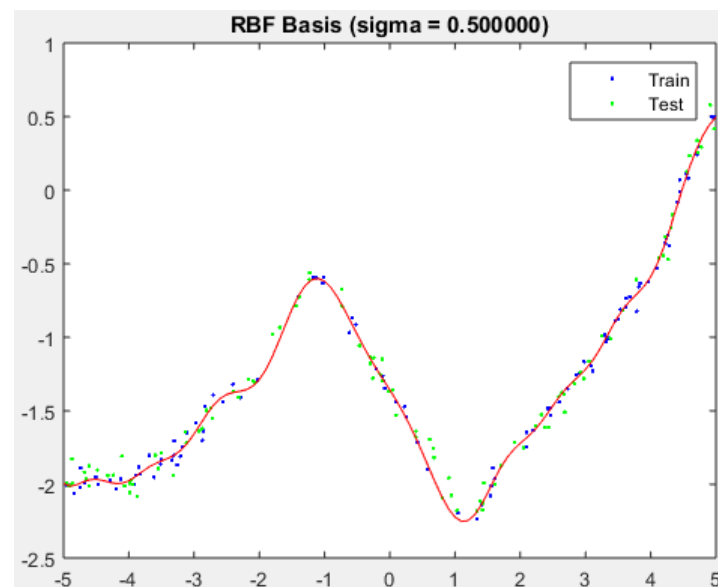
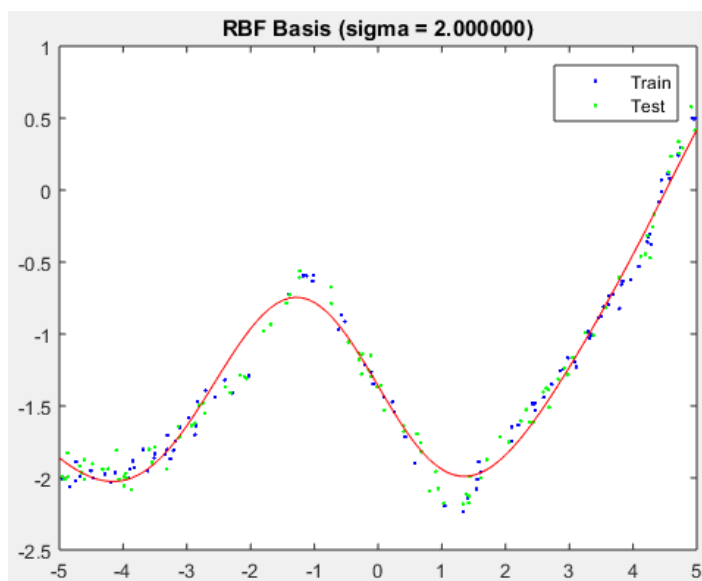
for each value of  $\lambda$  and  $\sigma$ :

- Compute  $Z$  on training data (and  $\sigma$ )
- Compute best  $v$ :  $v = (Z^T Z + \lambda I)^{-1} Z^T y$
- Compute  $\tilde{Z}$  on validation data (using train data distances)
- Make predictions  $\hat{y} = \tilde{Z} v$
- Compute validation error  $\|\hat{y} - \tilde{y}\|^2$



# RBFs, Regularization, and Validation

- A model that is hard to beat:
  - RBF basis with L2-regularization and cross-validation to choose  $\sigma$  and  $\lambda$ .
  - Flexible non-parametric basis, magic of regularization, and tuning for test error!



- Expensive at test time: needs distance to all training examples.
- Requires  $O(nd)$  storage for the training data.

# Hyper-Parameter Optimization

- In this setting we have **2 hyper-parameters** ( $\sigma$  and  $\lambda$ ).
- More complicated models have **even more hyper-parameters**.
  - This makes **searching all values expensive** (increases **over-fitting risk**).
- Leads to the problem of **hyper-parameter optimization**.
  - Try to efficiently find “best” hyper-parameters.
- Simplest approaches:
  - Exhaustive search: try all combinations among a fixed set of  $\sigma$  and  $\lambda$  values.
  - **Random search: try random values.**

distribution  
for  $\lambda$  and  $\sigma$   
and then it closes

# Hyper-Parameter Optimization

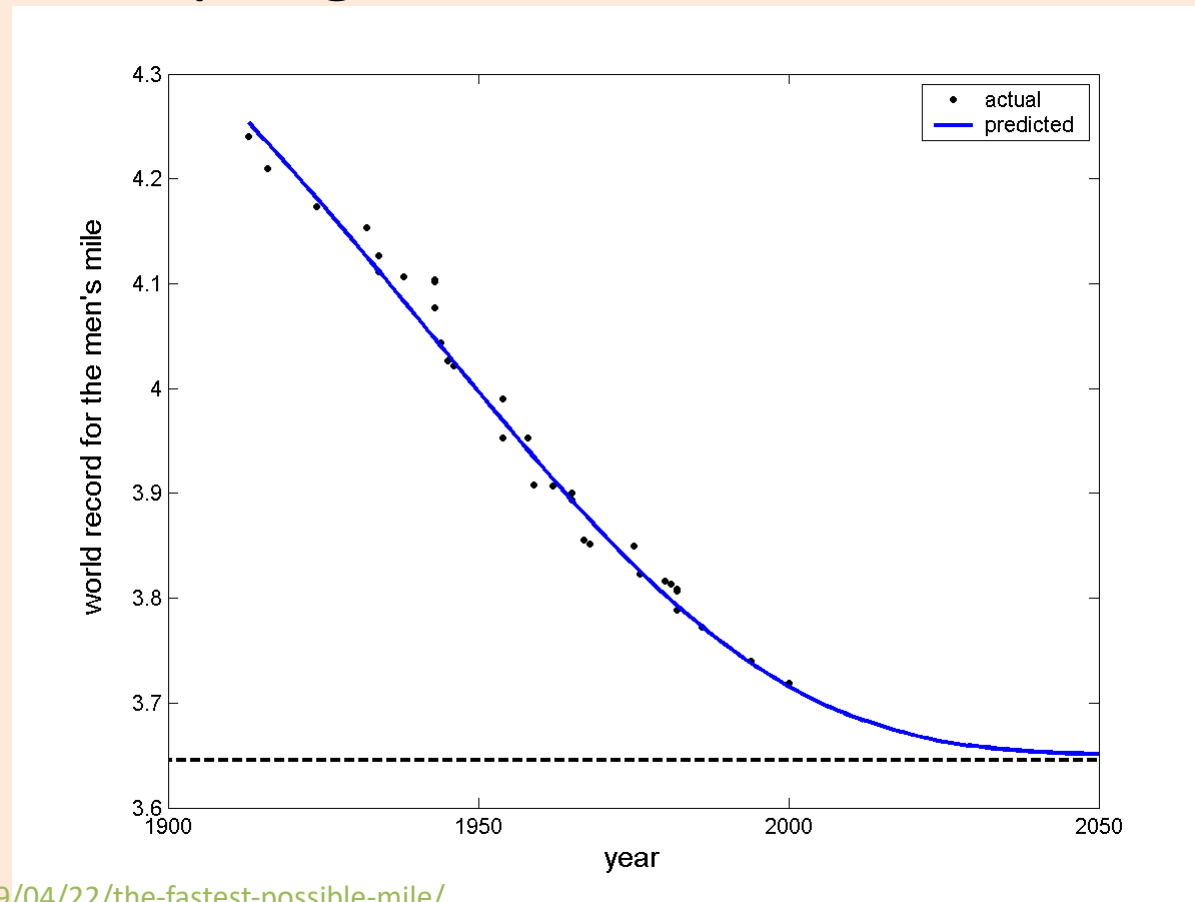
- Other common **hyper-parameter optimization** methods:
  - **Exhaustive search with pruning:**
    - If it “looks” like test error is getting worse as you decrease  $\lambda$ , stop decreasing it.
  - **Coordinate search:**
    - Optimize one hyper-parameter at a time, keeping the others fixed.
    - Repeatedly go through the hyper-parameters
  - **Stochastic local search:**
    - Generic global optimization methods (simulated annealing, genetic algorithms, and so on).
  - **Bayesian optimization:**
    - Use RBF regression to build **model of how hyper-parameters affect validation error**.
    - Try the best guess based on the model, then repeat.



Next Topic: Interpolation vs. Extrapolation

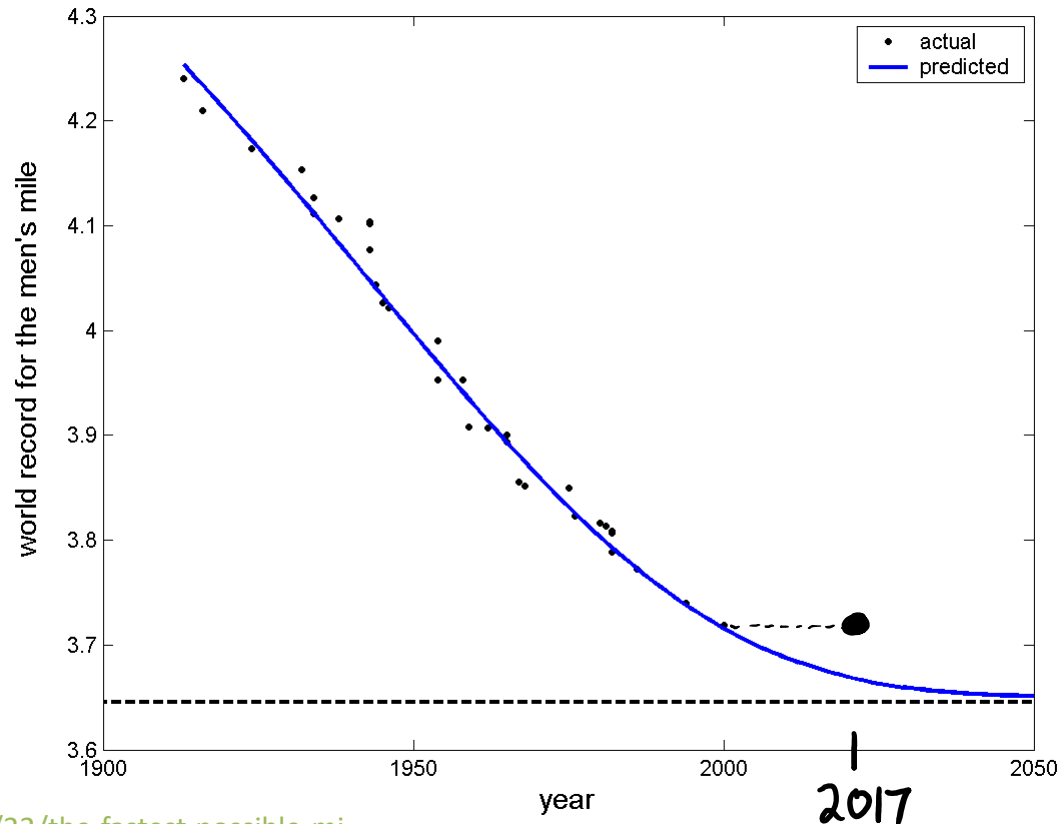
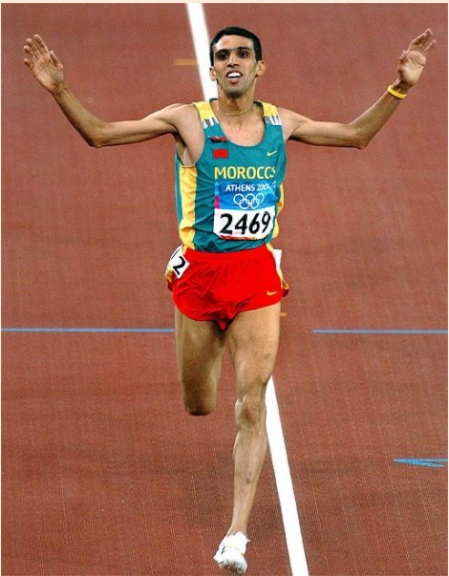
# Predicting the Future

- In principle, we can use any features  $x_i$  that we think are relevant.
- This makes it tempting to use **time** as a feature, and predict future.



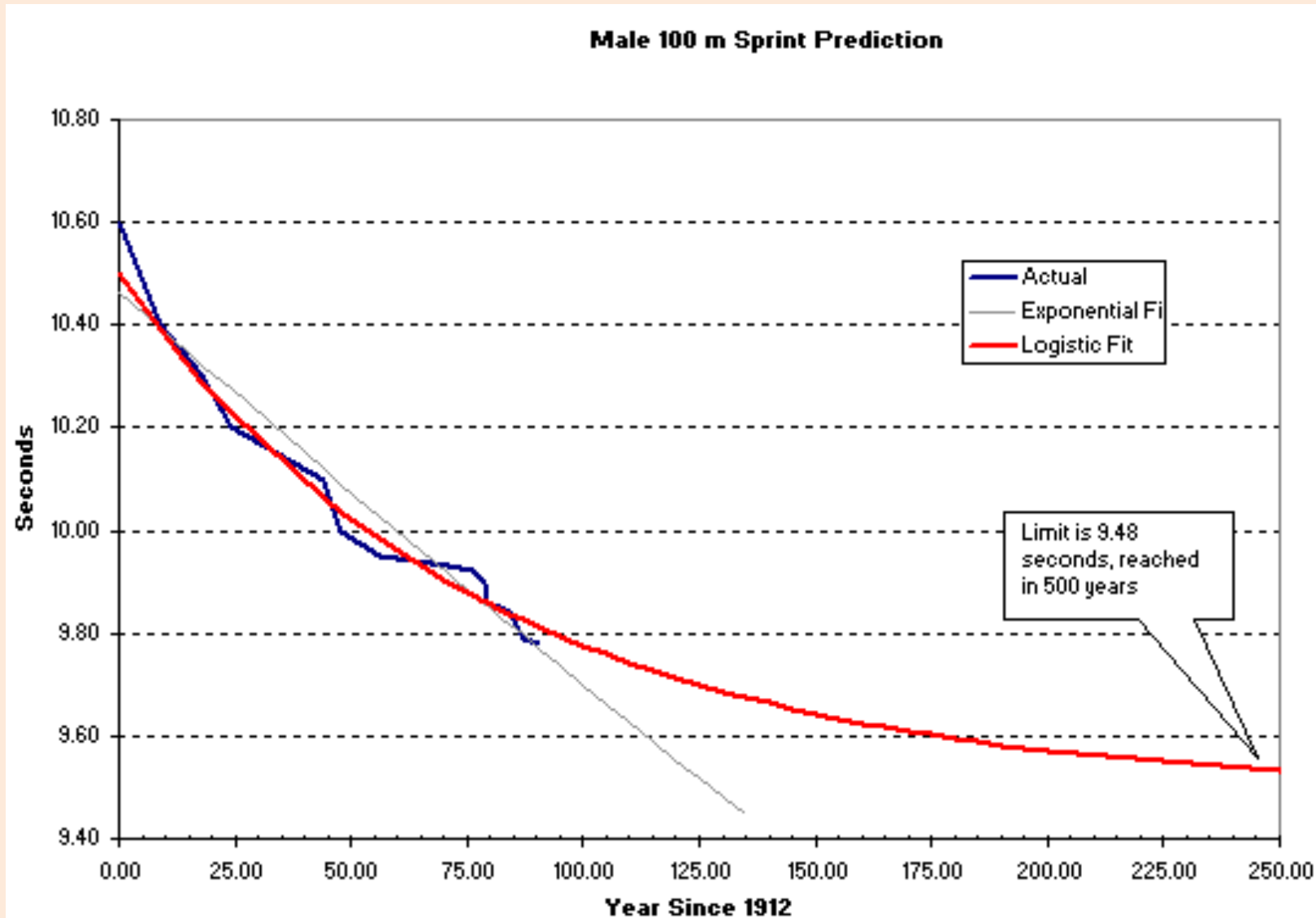
# Predicting the Future

- In principle, we can use any features  $x_i$  that we think are relevant.
- This makes it tempting to use **time as a feature**, and predict future.



We need to be  
Cautious about  
doing this.

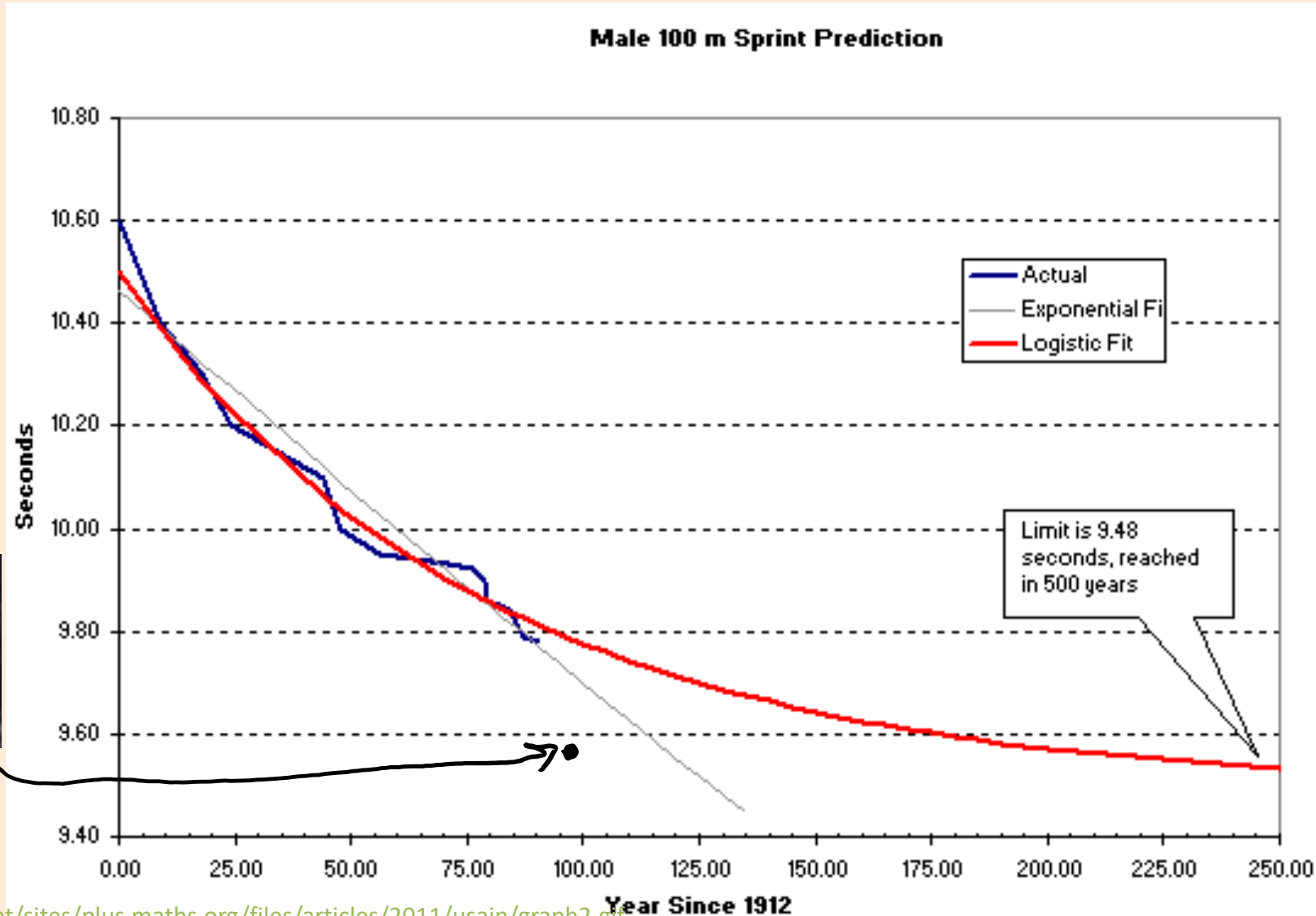
# Predicting 100m times 400 years in the future?



# Predicting 100m times 400 years in the future?



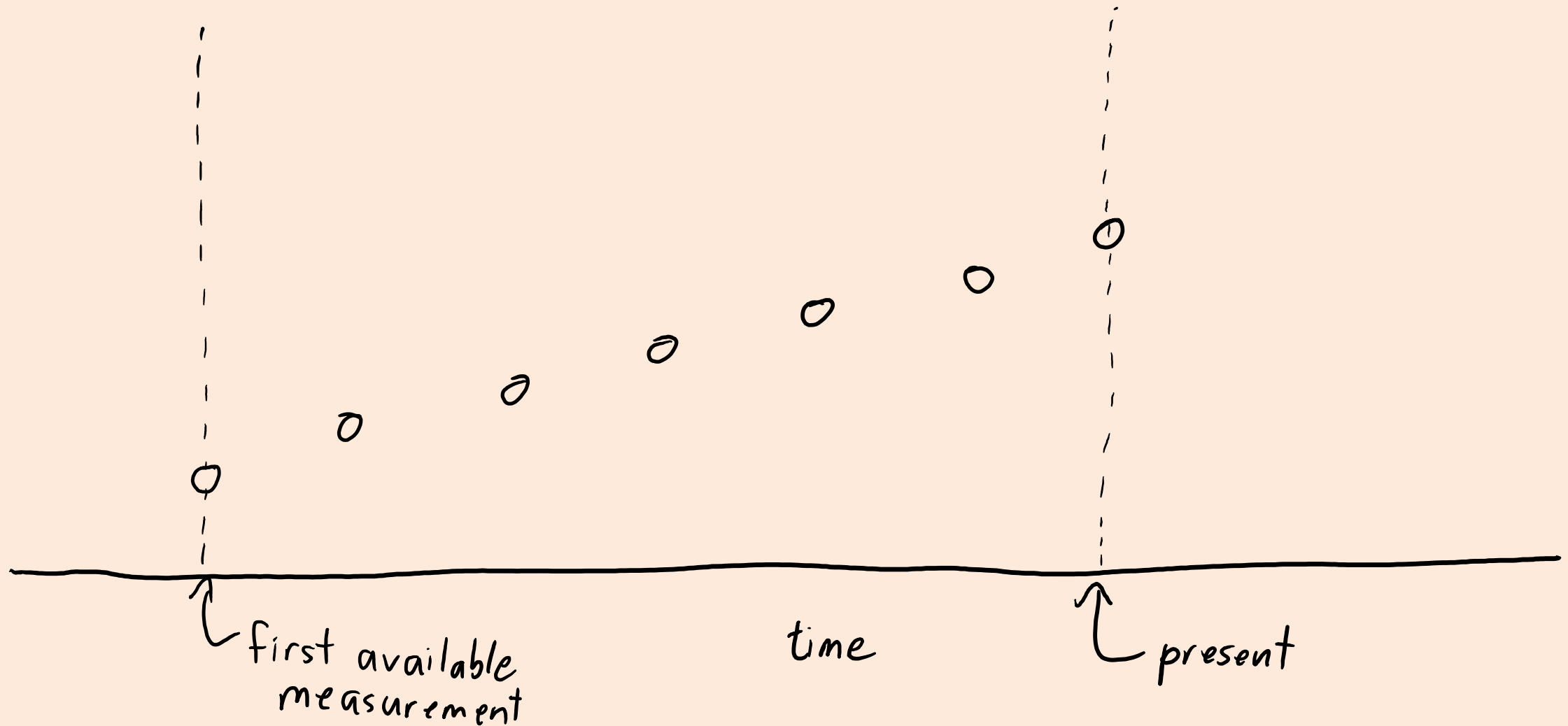
9.58



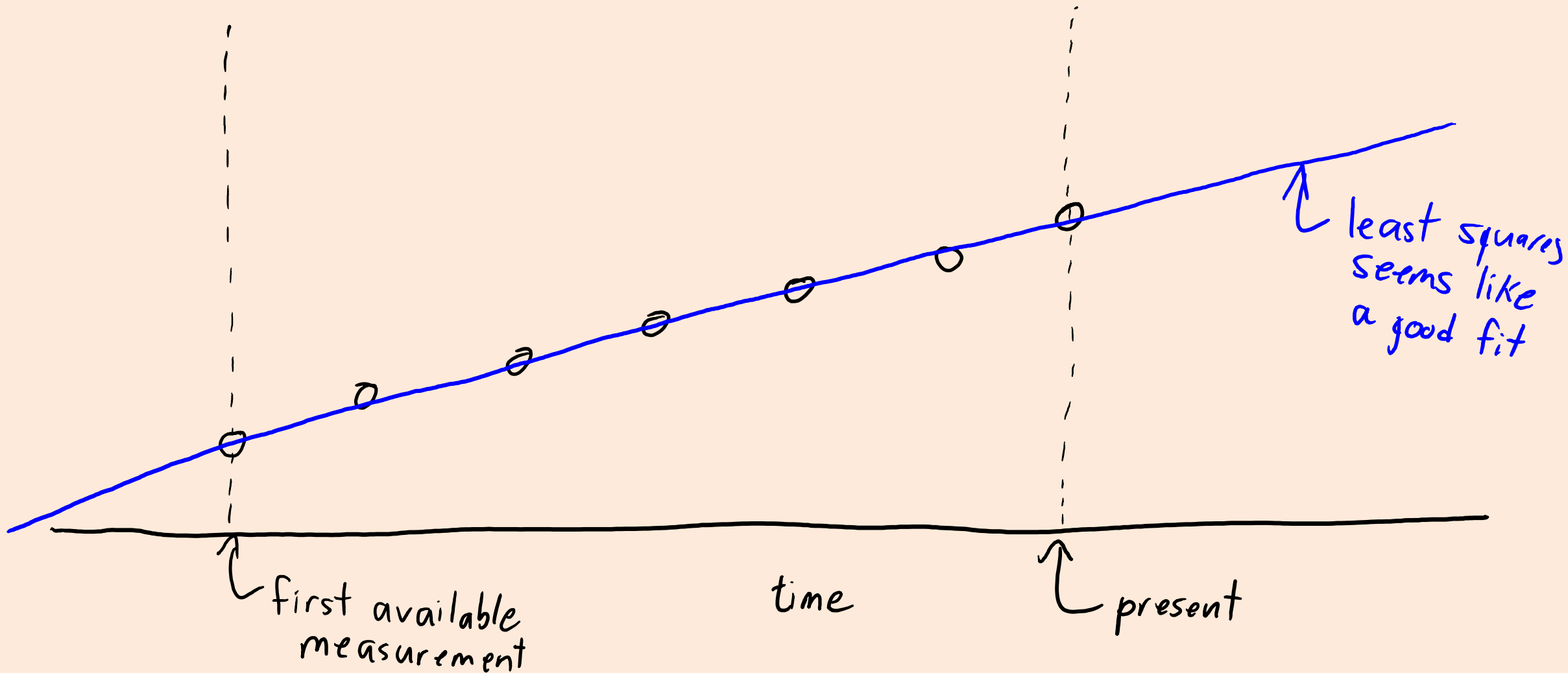
# Interpolation vs Extrapolation

- **Interpolation** is task of predicting “between the data points”.
  - Regression models are good at this if you have enough data and function is continuous.
- **Extrapolation** is task of prediction outside the range of the data points.
  - Without assumptions, regression models can be embarrassingly-bad at this.
- If you run the 100m regression models backwards in time:
  - They predict that **humans used to be really really slow!**
- If you run the 100m regression models forwards in time:
  - They might eventually predict arbitrarily-small 100m times.
  - The linear model actually predicts **negative times** in the future.
    - These time traveling races in 2060 should be pretty exciting!
- Some discussion here:
  - [http://callingbullshit.org/case\\_studies/case\\_study\\_gender\\_gap\\_running.html](http://callingbullshit.org/case_studies/case_study_gender_gap_running.html)

# No Free Lunch, Consistency, and the Future

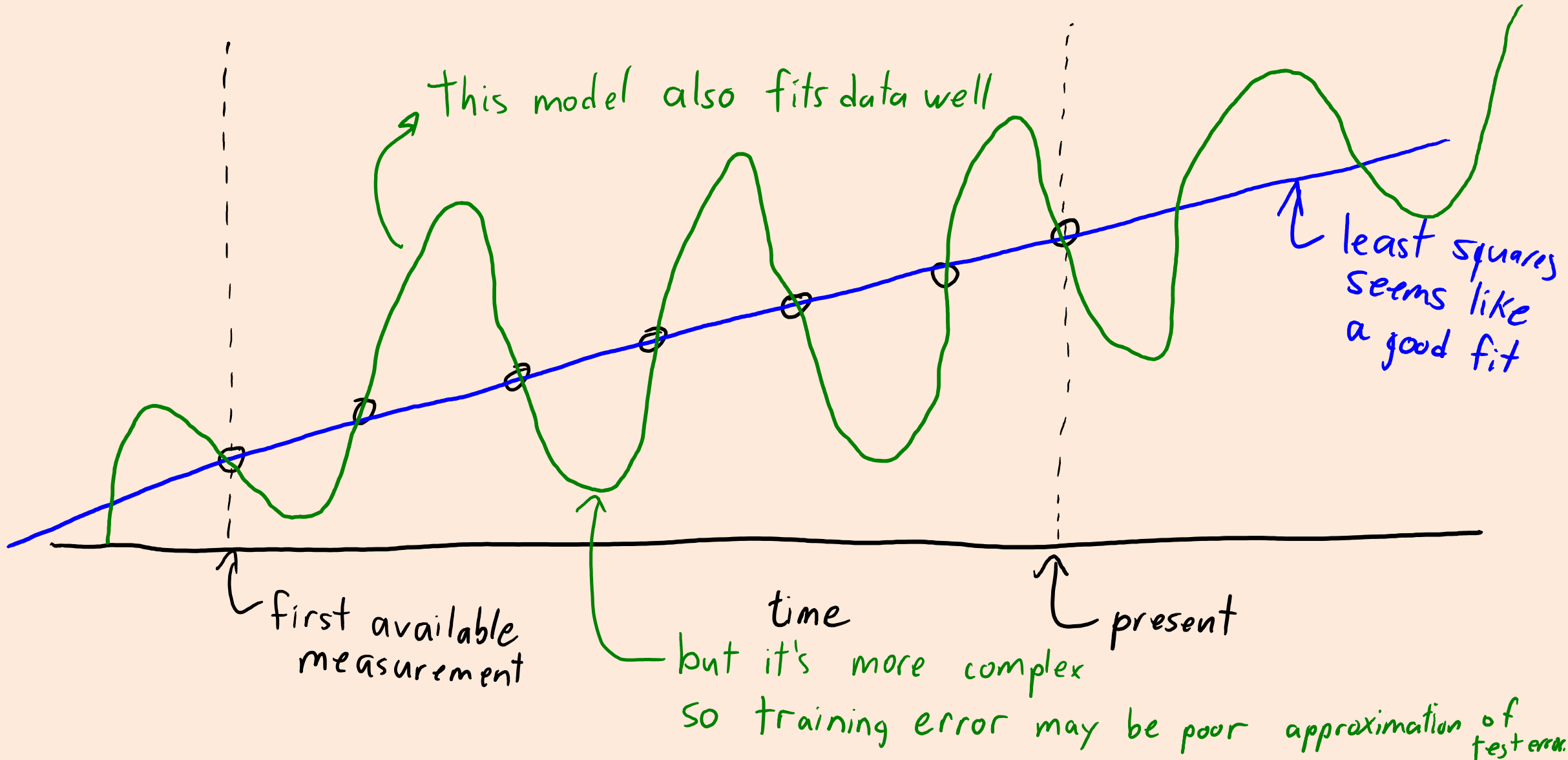


# No Free Lunch, Consistency, and the Future



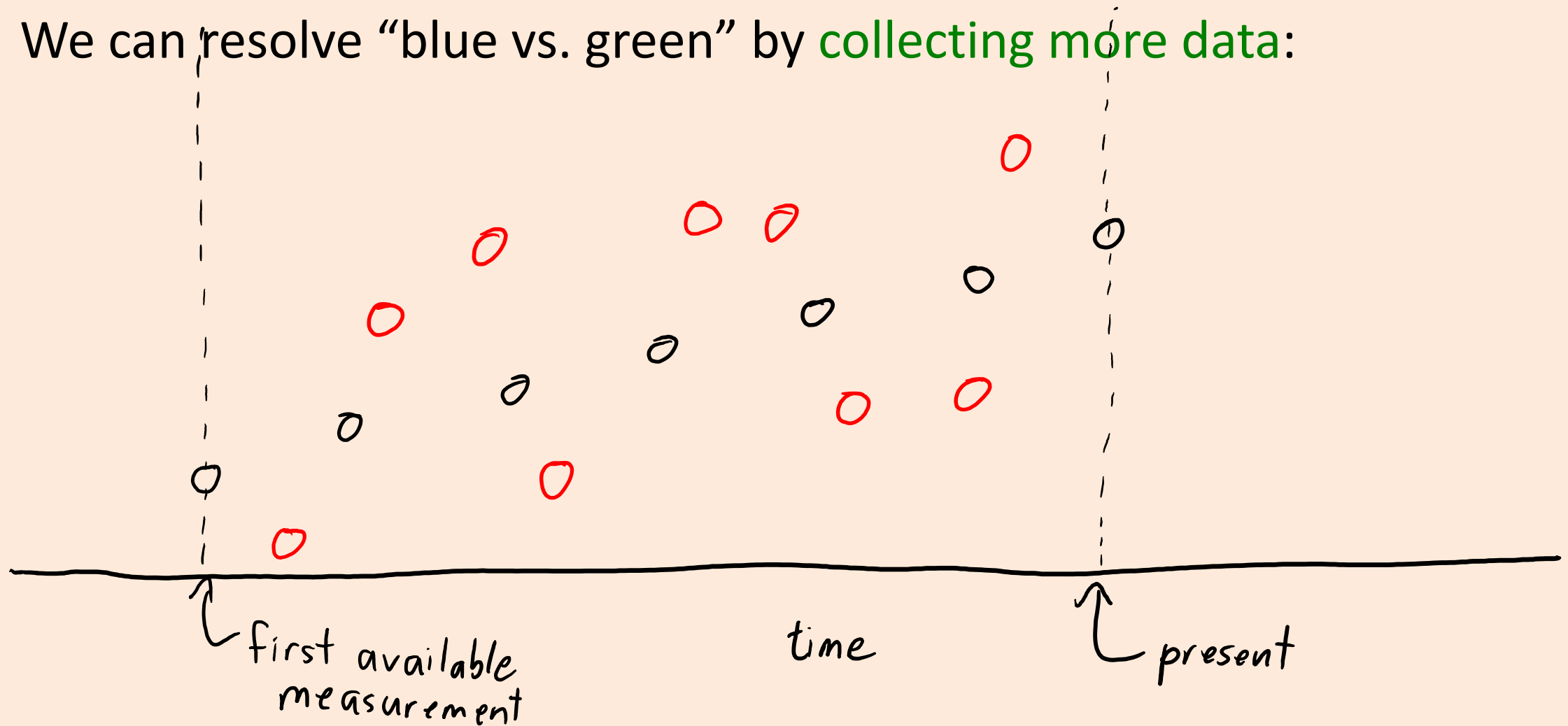


# No Free Lunch, Consistency, and the Future

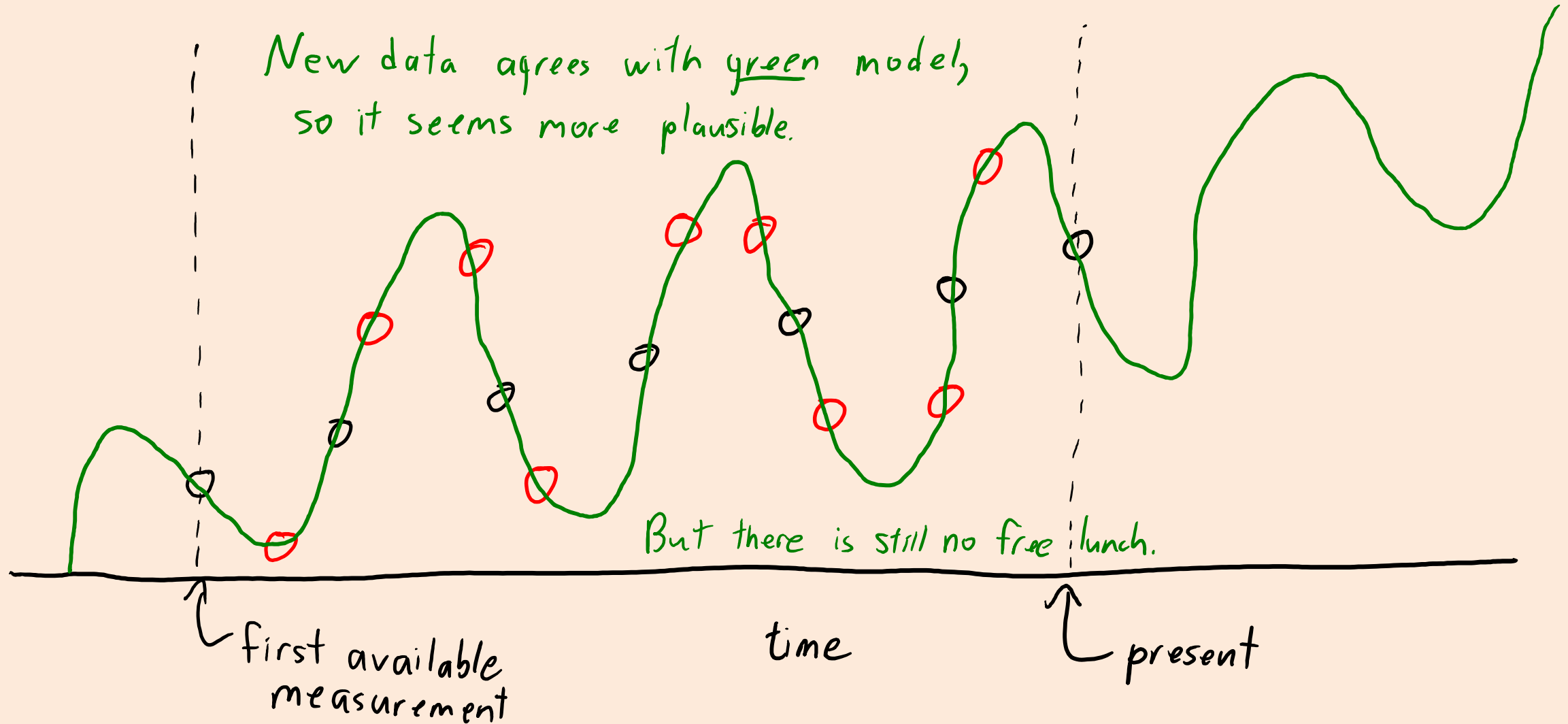


# No Free Lunch, Consistency, and the Future

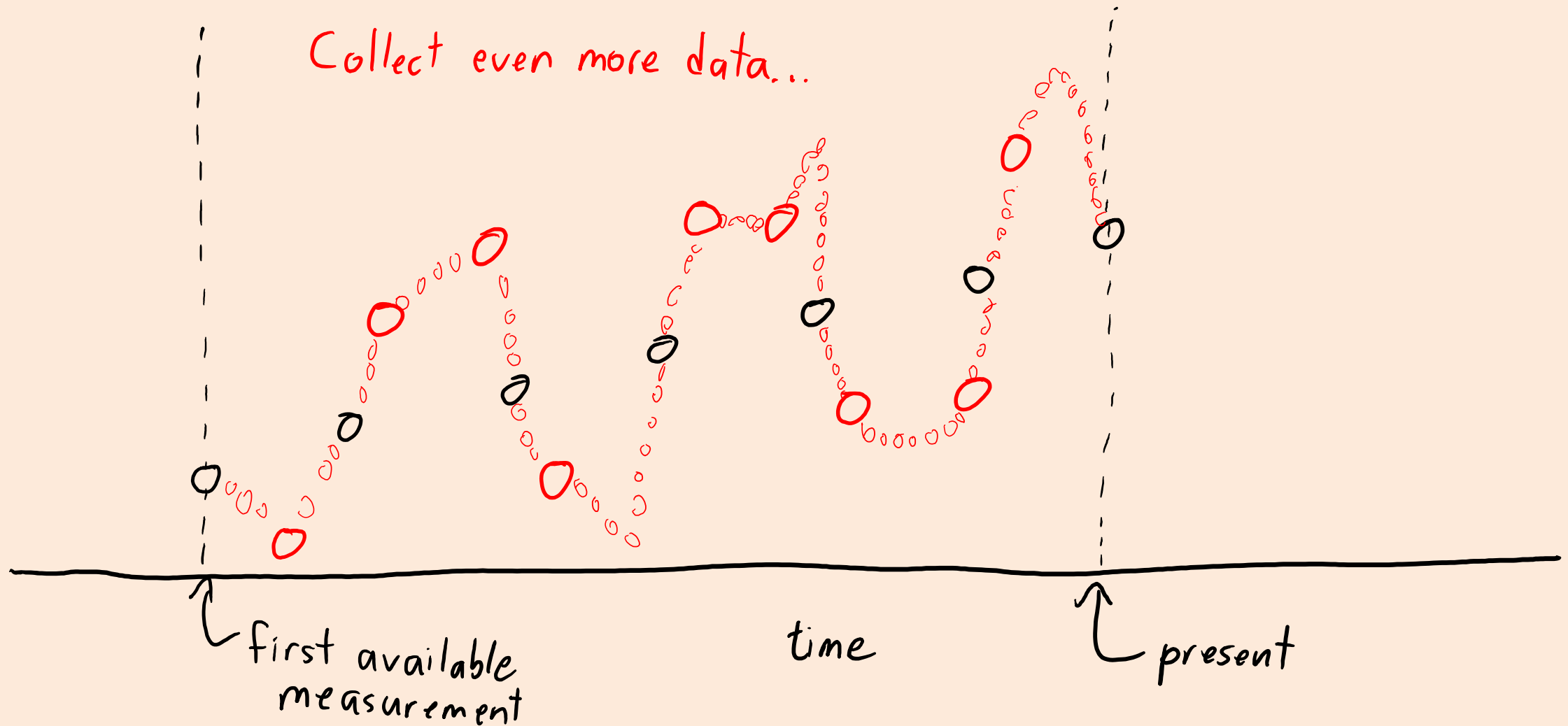
- We can resolve “blue vs. green” by **collecting more data**:



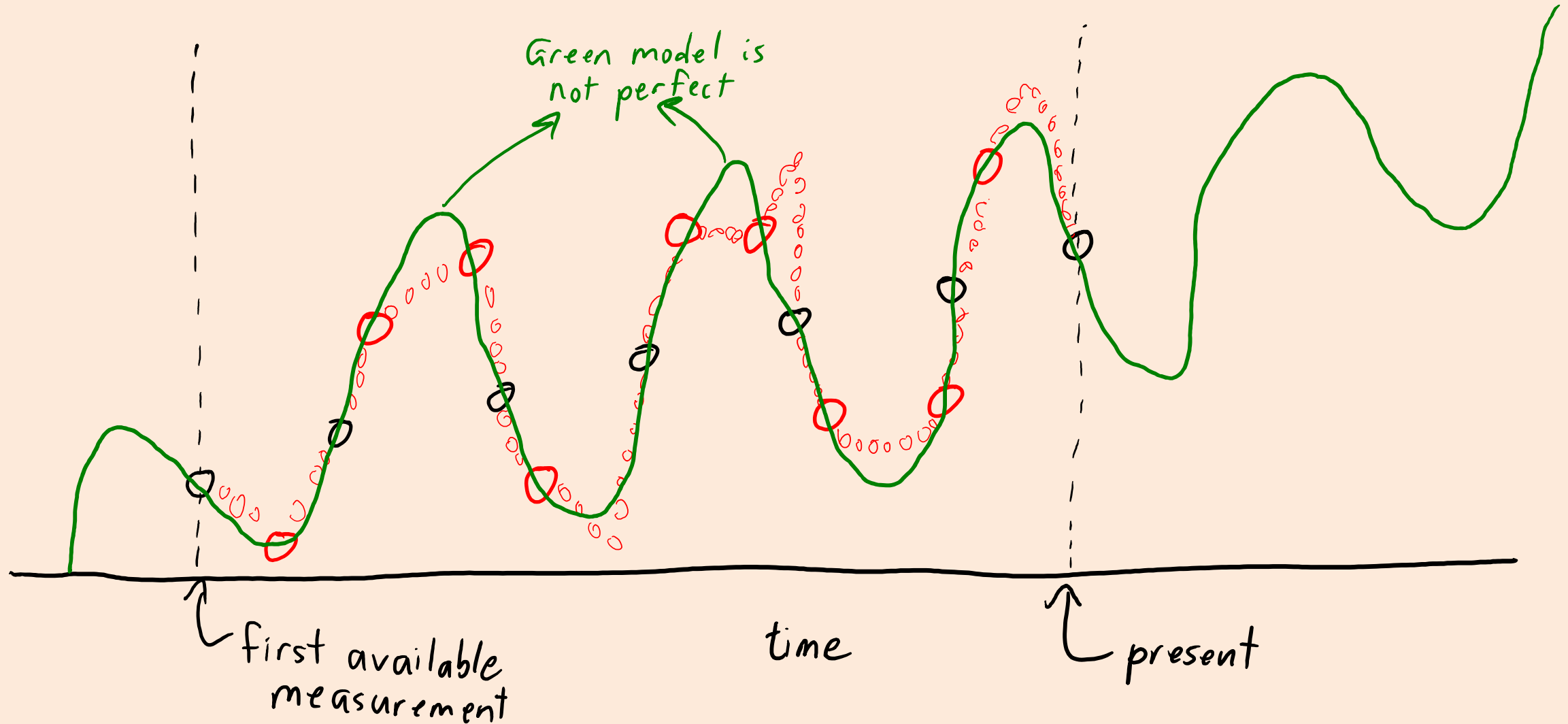
# No Free Lunch, Consistency, and the Future



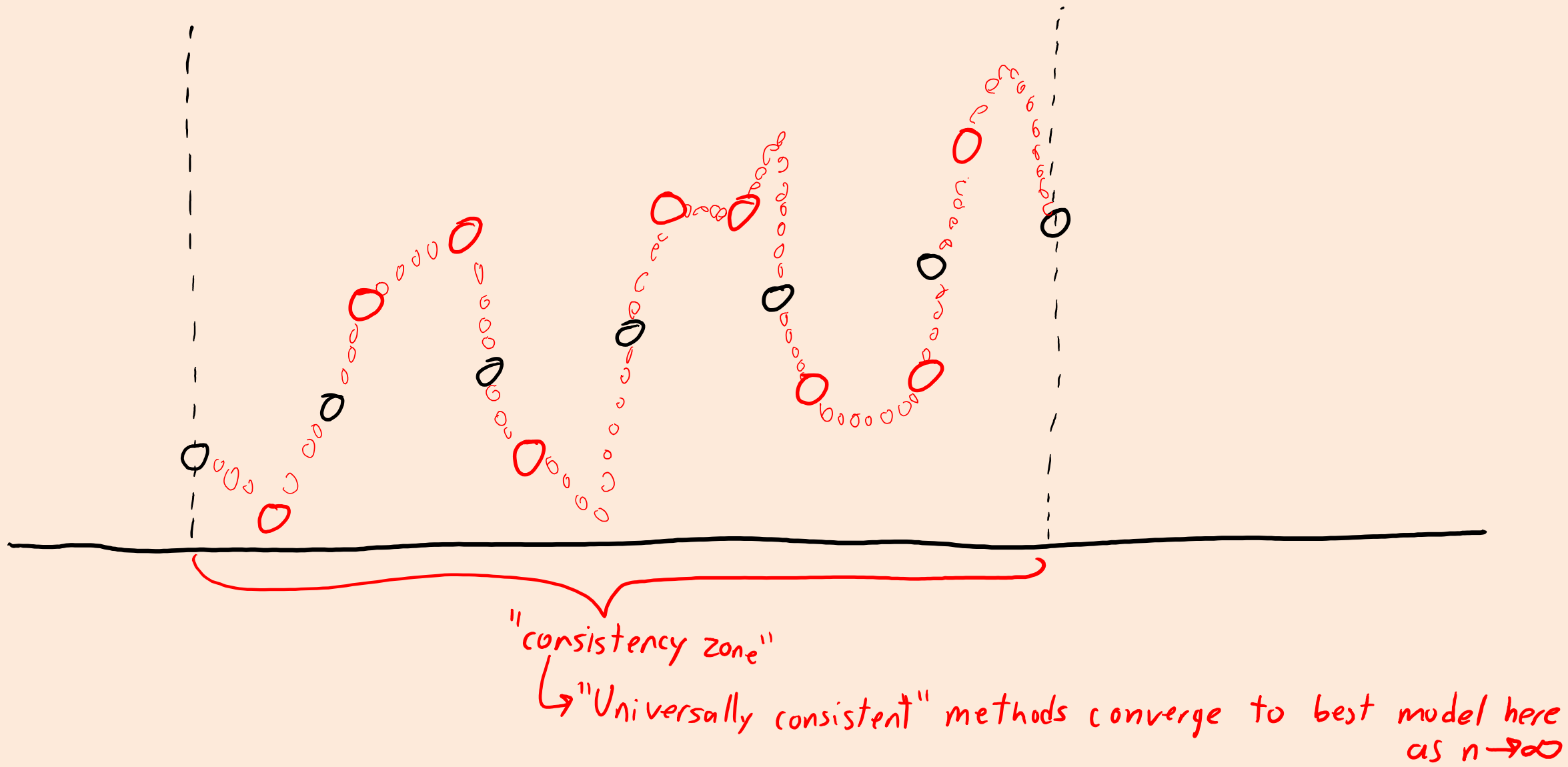
# No Free Lunch, Consistency, and the Future



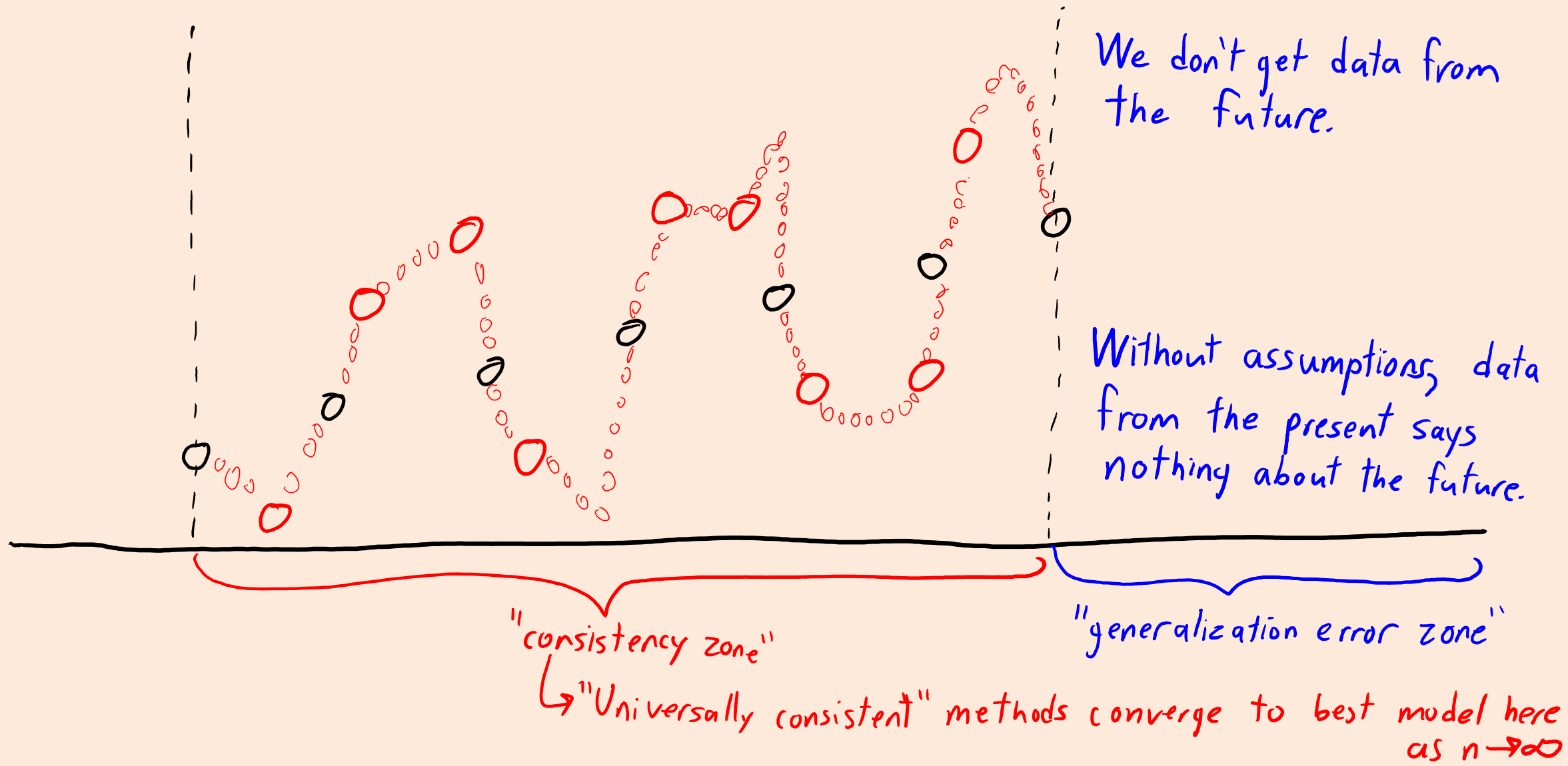
# No Free Lunch, Consistency, and the Future



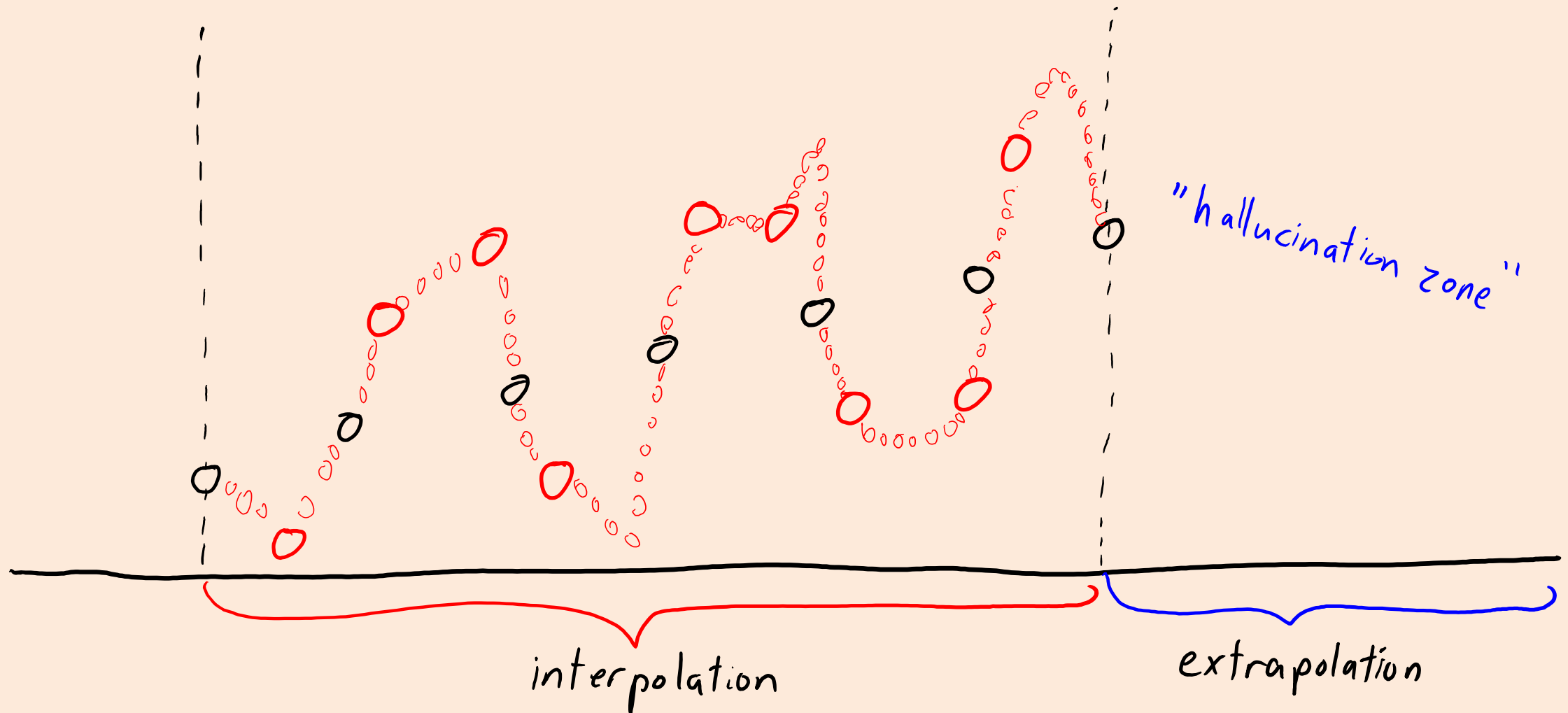
# No Free Lunch, Consistency, and the Future



# No Free Lunch, Consistency, and the Future



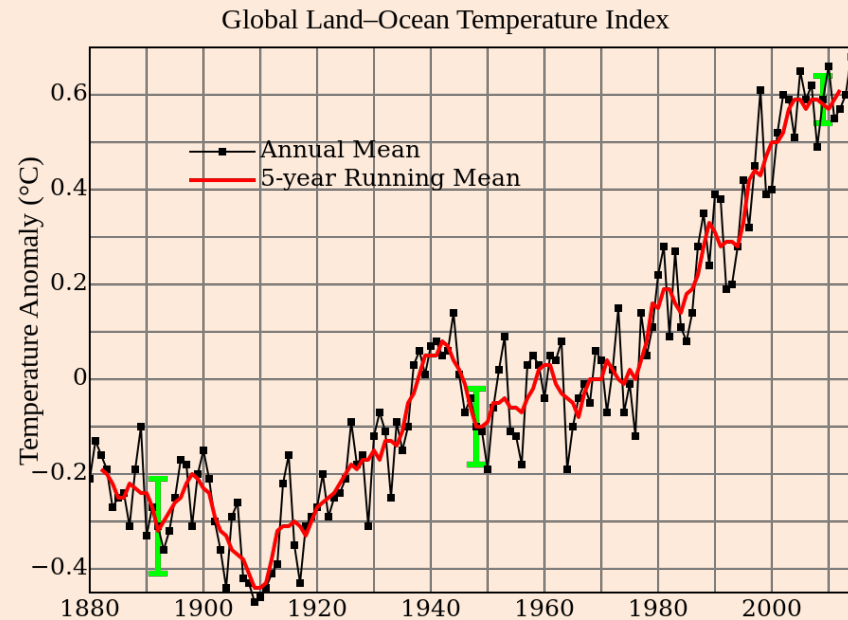
# No Free Lunch, Consistency, and the Future





# Discussion: Climate Models

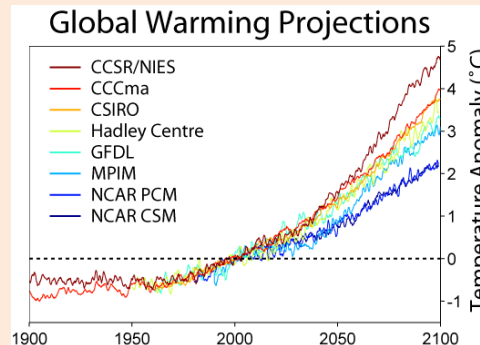
- Has Earth warmed up over last 100 years? (Consistency zone)
  - Data clearly says “yes”.



- Will Earth continue to warm over next 100 years? (generalization error)
  - We should be more skeptical about models that predict future events.

# Discussion: Climate Models

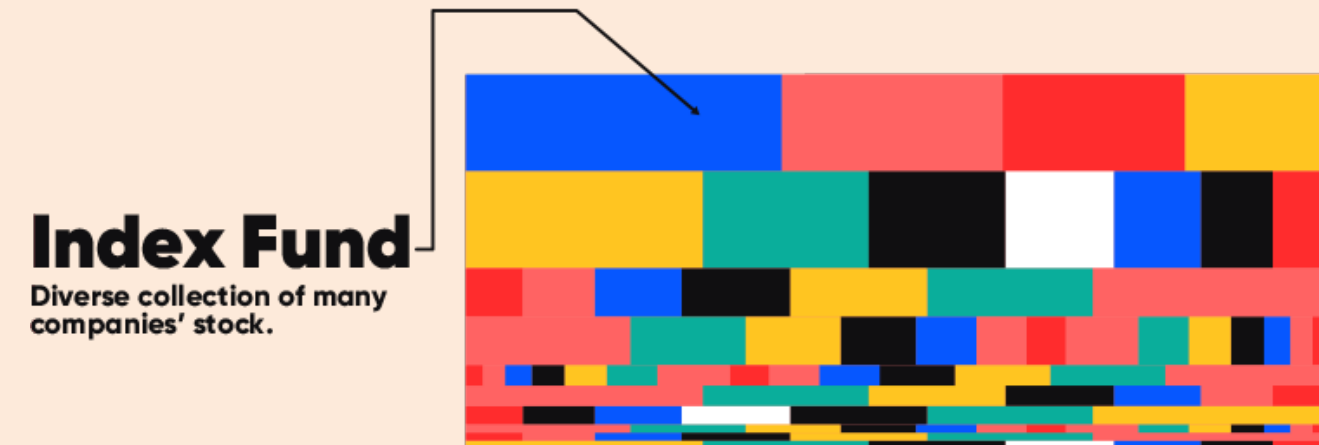
- So should we all become global warming skeptics?
- If we **average over models that overfit in \*independent\* ways, we expect the test error to be lower**, so this gives more confidence:



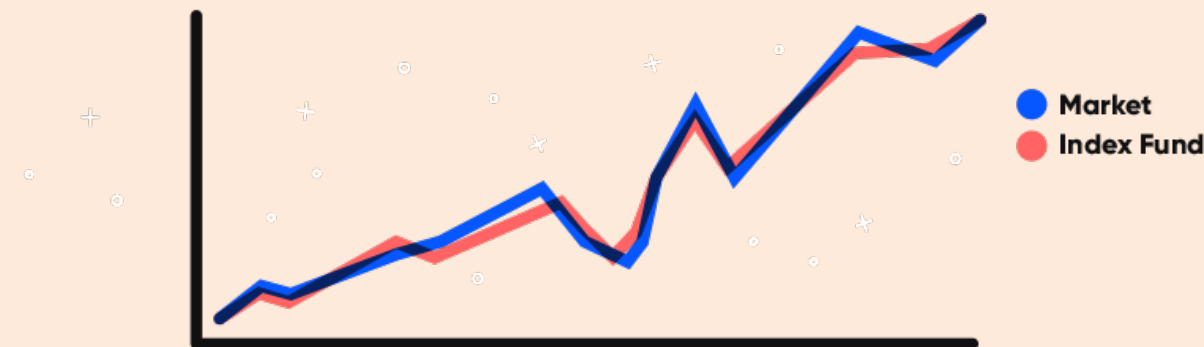
- We should be skeptical of individual models, but agreeing predictions made by models with different data/assumptions are more likely to be true.
- All the near-future predictions agree, so they are likely to be accurate.
  - And it's probably reasonable to assume fairly **continuous change** (no big “jumps”).
- Variance is higher further into future, so predictions are less reliable.
  - Relying more on assumptions and less on data.

# Index Funds: Ensemble Extrapolation for Investing

- Want to do **extrapolation when investing money**.
  - What will this be worth in the future?
- **Index funds** can be viewed as an ensemble method for investing.
  - For example, buy stock in top 500 companies proportional to value.
  - Tries to follow average price increase/decrease.



**GOAL = Match the Market (Index)**



- This simple investing strategy **outperforms most managed funds**.

Next Topic: L1-Regularization

# Previously: Search and Score

- We talked about **search and score** for **feature selection**:
  - Define a “score” and “search” for features with the best score.
- Usual scores **count the number of non-zeroes** (“**L0-norm**”):

$$f(w) = \frac{1}{2} \|Xw - y\|^2 + \underbrace{\lambda \|w\|_0}_{\text{number of non-zeroes in 'w'}}$$

- But it's **hard to find the 'w'** minimizing this objective.
- We discussed **forward selection**, but requires **fitting  $O(d^2)$  models**.

# Previously: Search and Score

- What if we want to pick among millions or billions of variables?
- If 'd' is large, forward selection is too slow:
  - For least squares, need to fit  $O(d^2)$  models at cost of  $O(nd^2 + d^3)$ .
  - Total cost  $O(nd^4 + d^5)$ , and even if you are clever still costs  $O(nd^2 + d^4)$ .
- The situation is worse if we are not using basic least squares:
  - For robust regression, need to run gradient descent  $O(d^2)$  times.
  - With regularization, need to search for lambda  $O(d^2)$  times.

# L1-Regularization

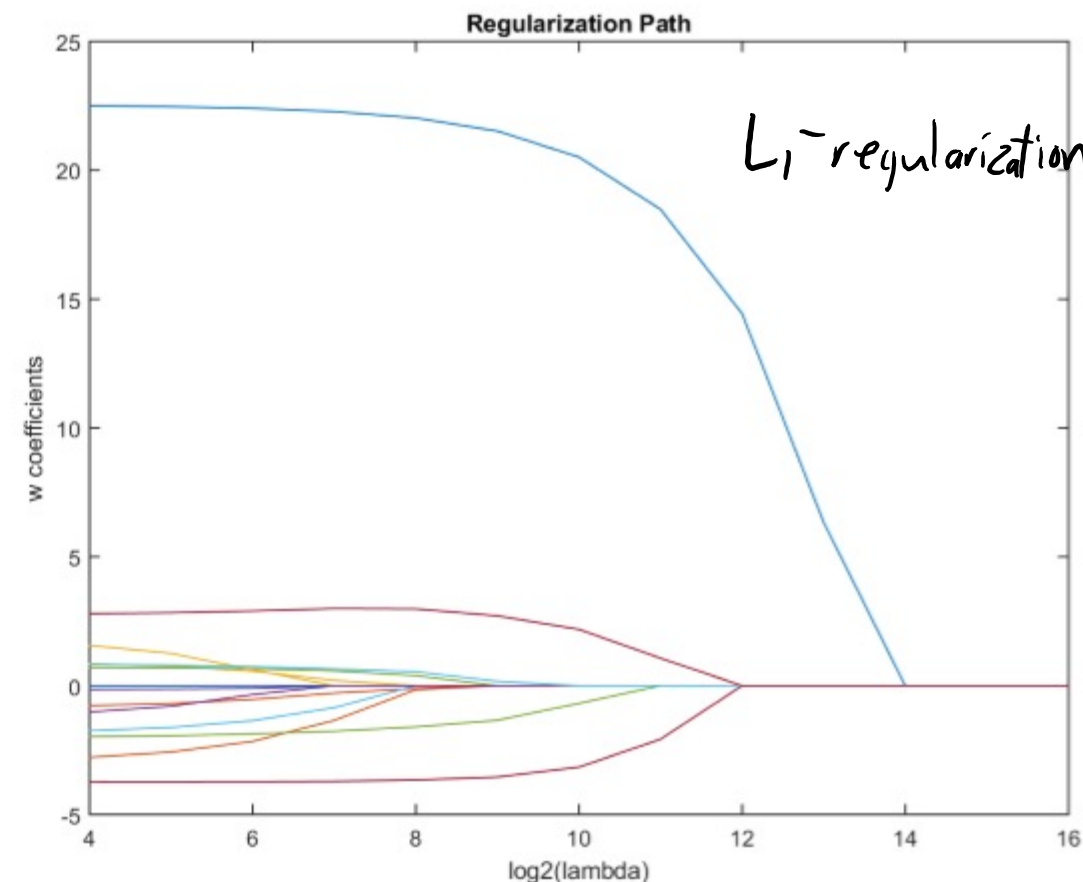
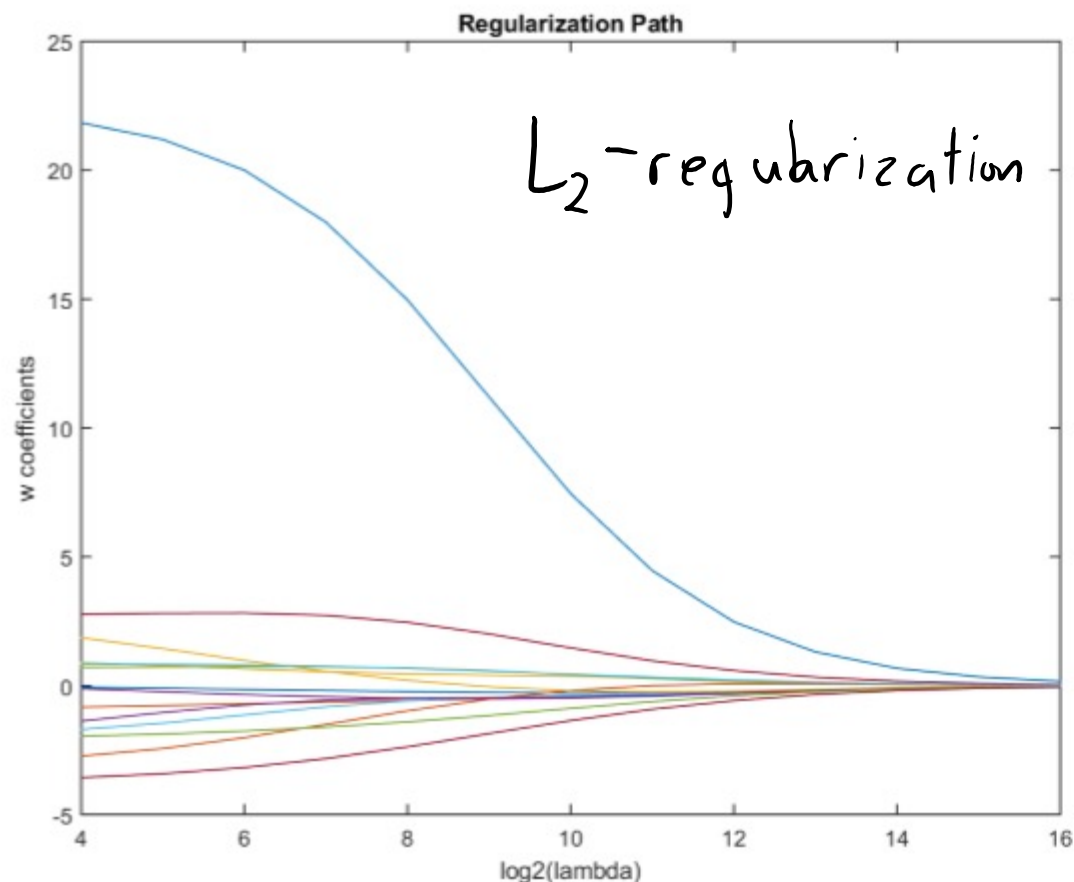
- Instead of L0- or L2-norm, consider regularizing by the L1-norm:

$$f(w) = \frac{1}{2} \|Xw - y\|^2 + \lambda \|w\|_1$$

- Like L2-norm, it's convex and improves our test error.
- Like L0-norm, it encourages elements of 'w' to be exactly zero.
- L1-regularization simultaneously regularizes and selects features.
  - Very fast alternative to search and score.
  - Sometimes called “LASSO” regularization (or “basis pursuit denoising”).

# L2-Regularization vs. L1-Regularization

- Regularization path of  $w_i$  values as ' $\lambda$ ' varies:



- L1-Regularization sets values to exactly 0 (next slides explore why).



# Regularizers and Sparsity

- L1-regularization gives sparsity but L2-regularization does not.
  - But don't they both shrink variables towards zero?
- What is the penalty for setting  $w_j = 0.00001$ ?
- L0-regularization: penalty of  $\lambda$ .
  - A constant penalty for any non-zero value.
  - Encourages you to set  $w_j$  exactly to zero, but otherwise doesn't care if  $w_j$  is small or not.
- L2-regularization: penalty of  $(\lambda/2)(0.00001)^2 = 0.0000000005\lambda$ .
  - The penalty gets smaller as you get closer to zero.
  - L2-regularization prefers to shrink larger values of  $w_j$  rather than shrink that are  $w_j$  close to 0.
- L1-regularization: penalty of  $\lambda|0.00001| = 0.00001\lambda$ .
  - The penalty stays is proportional to how far away  $w_j$  is from zero.
  - There is still something to be gained from making a tiny value exactly equal to 0.

sparsity... [0, 0, 1, 1, 0, 0]  
feature selection

Robust to outliers

# L2-Regularization vs. L1-Regularization

- L2-Regularization:

- Insensitive to changes in data.
- Decreased variance:
  - Lower test error.
- Closed-form solution.
- Solution is unique.
- All ' $w_j$ ' tend to be non-zero.
- Can learn with *linear* number of irrelevant features.
  - E.g., only  $O(d)$  relevant features.

Handwritten notes illustrating L2 regularization:

$$y = x_1 + 0 \cdot x_2$$

$$y = 0.5x_1 + 0.5x_2$$

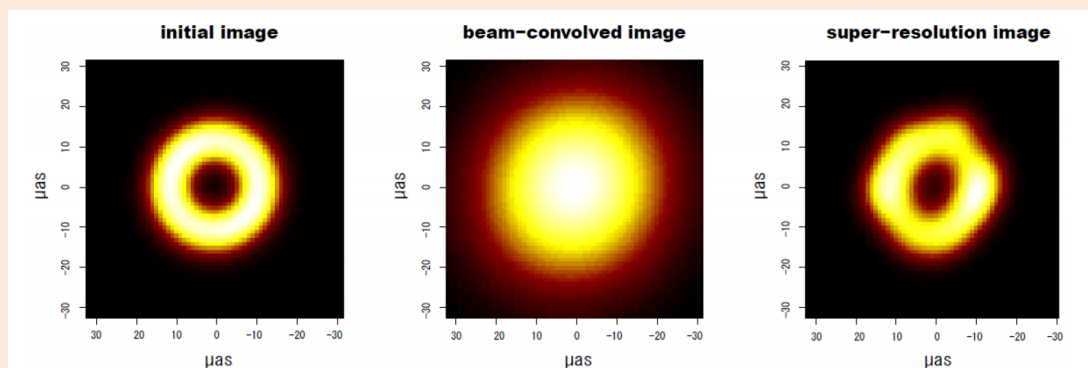
The second equation is circled in red. A red arrow labeled  $L_2$  points to the circle. Below the circle, the expression  $y = 0.5x_1 + 0.5x_2$  is written again, with a red arrow pointing to the circle.

- L1-Regularization:

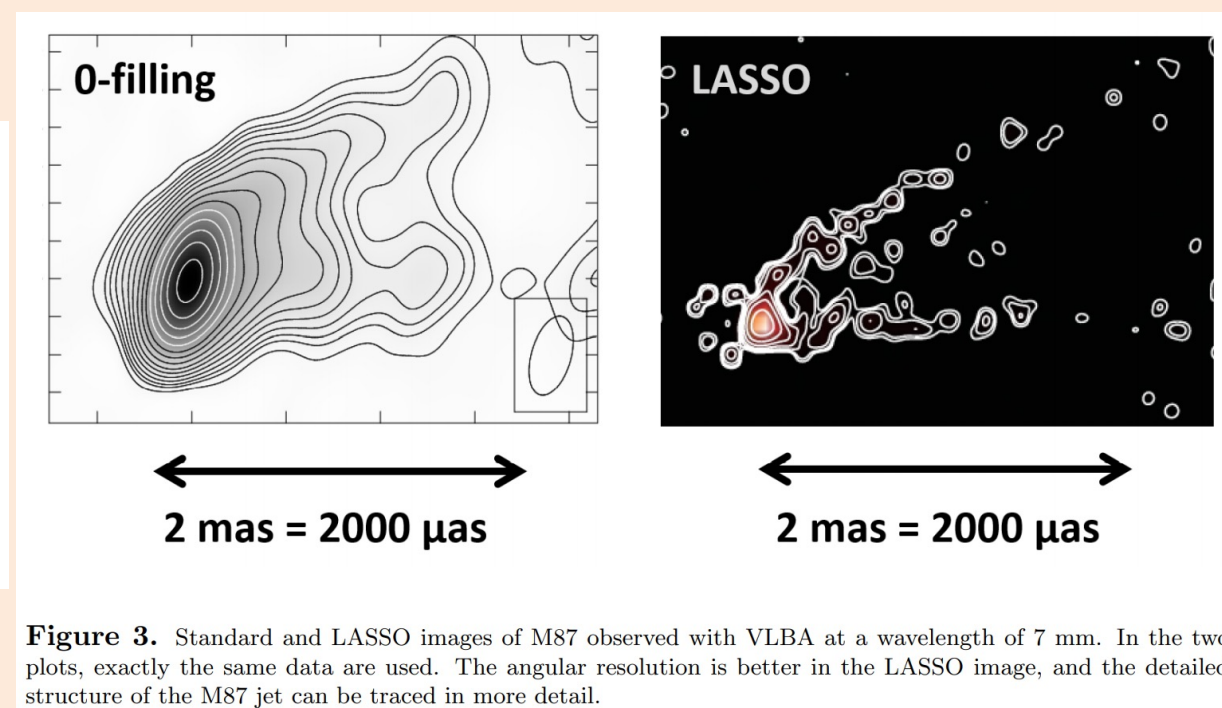
- Insensitive to changes in data.
  - Decreased variance:
    - Lower test error.
  - Requires iterative solver.
  - Solution is not unique.
  - Many ' $w_j$ ' tend to be zero.
  - Can learn with **exponential** number of irrelevant features.
    - E.g., only  $O(\log(d))$  relevant features.
- [Paper on this result by Andrew Ng](#)

# L1-Regularization Applications

- Used to give super-resolution in imaging black holes.
  - Sparsity arises in a particular basis.



**Figure 2.** Simulated images of M87. From left to right, the initial model, the image with 0-filling, and the image with LASSO. Improvement of resolution in the LASSO image is significant.



**Figure 3.** Standard and LASSO images of M87 observed with VLBA at a wavelength of 7 mm. In the two plots, exactly the same data are used. The angular resolution is better in the LASSO image, and the detailed structure of the M87 jet can be traced in more detail.

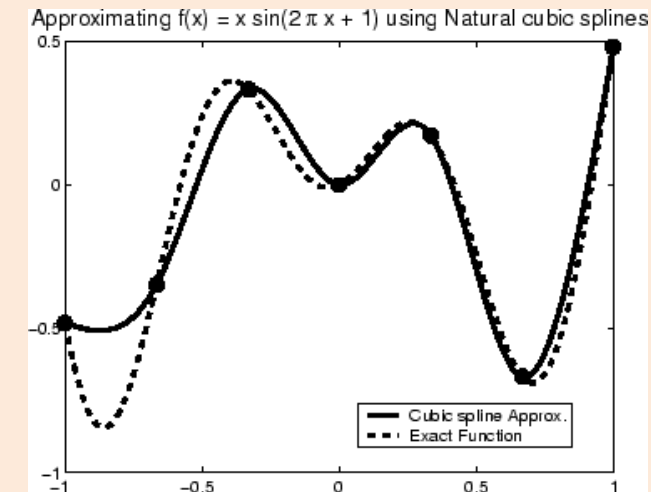
- Another application:
  - Use L1-regularization with Gaussian RBFs to reduce prediction time.

# Summary

- Radial basis functions:
  - Non-linear transforms based on distances to training points.
  - Non-parametric bases that can model any function.
  - But prediction is slow since with 'n' training examples you have 'n' features.
- Interpolation vs. Extrapolation:
  - Machine learning with large 'n' is good at predicting “between the data”.
  - Without assumptions, can be arbitrarily bad “away from the data”.
- L1-regularization:
  - Simultaneous regularization and feature selection.
  - Robust to having lots of irrelevant features.
- Next time: are we really going to use regression for classification?

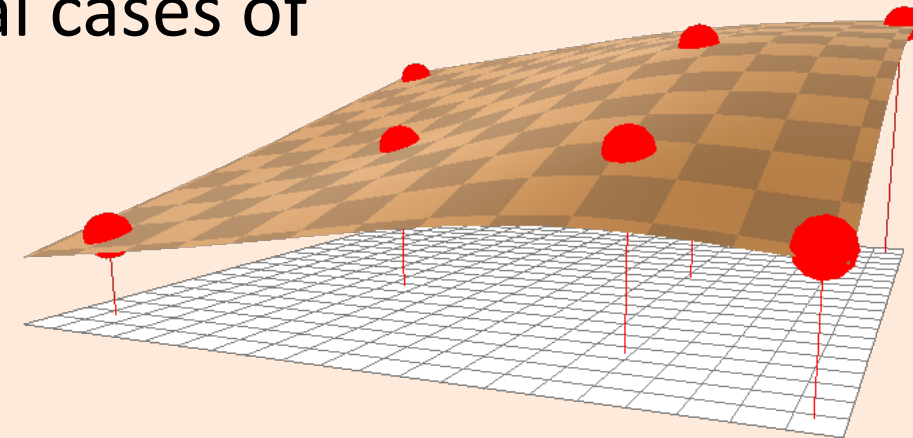
# Splines in 1D

- For 1D interpolation, alternative to polynomials/RBFs are splines:
  - Use a polynomial in the region between each data point.
  - Constrain some derivatives of the polynomials to yield a unique solution.
- Most common example is cubic spline:
  - Use a degree-3 polynomial between each pair of points.
  - Enforce that  $f'(x)$  and  $f''(x)$  of polynomials agree at all point.
  - “Natural” spline also enforces  $f''(x) = 0$  for smallest and largest  $x$ .
- Non-trivial fact: natural cubic splines are sum of:
  - Y-intercept.
  - Linear basis.
  - RBFs with  $g(\varepsilon) = \varepsilon^3$ .
    - Different than Gaussian RBF because it *increases with distance*.



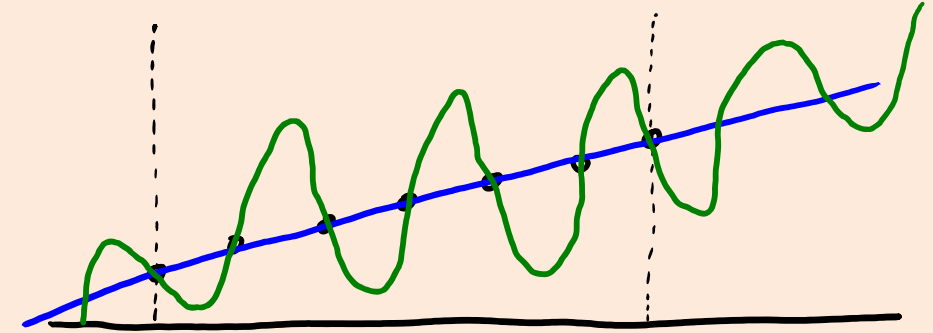
# Splines in Higher Dimensions

- Splines generalize to higher dimensions if data lies on a grid.
  - Many methods exist for grid-structured data (linear, cubic, splines, etc.).
  - For more general (“scattered”) data, there isn’t a natural generalization.
- Common 2D “scattered” data interpolation is thin-plate splines:
  - Based on curve made when bending sheets of metal.
  - Corresponds to RBFs with  $g(\epsilon) = \epsilon^2 \log(\epsilon)$ .
- Natural splines and thin-plate splines: special cases of “polyharmonic” splines:
  - Less sensitive to parameters than Gaussian RBF.



# Ockham's Razor vs. No Free Lunch

- **Ockham's razor** is a problem-solving principle:
  - “Among competing hypotheses, the one with the fewest assumptions should be selected.”
  - Suggests we should **select linear model**.
- **Fundamental trade-off:**
  - If same training error, pick model less likely to overfit.
  - Formal version of Occam's problem-solving principle.
  - Also suggests we should **select linear model**.
- **No free lunch theorem:**
  - There *exists possible datasets* where you should select the **green model**.



# Regularizers and Sparsity

- L1-regularization gives sparsity but L2-regularization doesn't.
  - But don't they both shrink variables to zero?
- Consider problem where 3 vectors can get minimum training error:

$$w^1 = \begin{bmatrix} 100 \\ 0.02 \end{bmatrix} \quad w^2 = \begin{bmatrix} 100 \\ 0 \end{bmatrix} \quad w^3 = \begin{bmatrix} 99.99 \\ 0.02 \end{bmatrix}$$

- Without regularization, we could choose any of these 3.
  - They all have same error, so regularization will “break tie”.
- With L0-regularization, we would choose  $w^2$ :

$$\|w^1\|_0 = 2 \quad \|w^2\|_0 = 1 \quad \|w^3\|_0 = 2$$



# Regularizers and Sparsity

- L1-regularization gives sparsity but L2-regularization doesn't.
  - But don't they both shrink variables to zero?
- Consider problem where 3 vectors can get minimum training error:

$$w^1 = \begin{bmatrix} 100 \\ 0.02 \end{bmatrix} \quad w^2 = \begin{bmatrix} 100 \\ 0 \end{bmatrix} \quad w^3 = \begin{bmatrix} 99.99 \\ 0.02 \end{bmatrix}$$

- With L2-regularization, we would choose  $w^3$ :

$$\begin{aligned} \|w^1\|^2 &= 100^2 + 0.02^2 \\ &= 10000.0004 \end{aligned} \quad \begin{aligned} \|w^2\|^2 &= 100^2 + 0^2 \\ &= 10000 \end{aligned} \quad \begin{aligned} \|w^3\|^2 &= 99.99^2 + 0.02^2 \\ &= 9998.0005 \end{aligned}$$

- L2-regularization focuses on decreasing largest (makes  $w_j$  similar).

# Regularizers and Sparsity

- L1-regularization gives sparsity but L2-regularization doesn't.
  - But don't they both shrink variables to zero?
- Consider problem where 3 vectors can get minimum training error:

$$w^1 = \begin{bmatrix} 100 \\ 0.02 \end{bmatrix} \quad w^2 = \begin{bmatrix} 100 \\ 0 \end{bmatrix} \quad w^3 = \begin{bmatrix} 99.99 \\ 0.02 \end{bmatrix}$$

- With L1-regularization, we would choose  $w^2$ :

$$\begin{aligned} \|w^1\|_1 &= 100 + 0.02 \\ &= 100.02 \end{aligned} \quad \begin{aligned} \|w^2\|_1 &= 100 + 0 \\ &= 100 \end{aligned} \quad \begin{aligned} \|w^3\|_1 &= 99.99 + 0.02 \\ &= 100.01 \end{aligned}$$

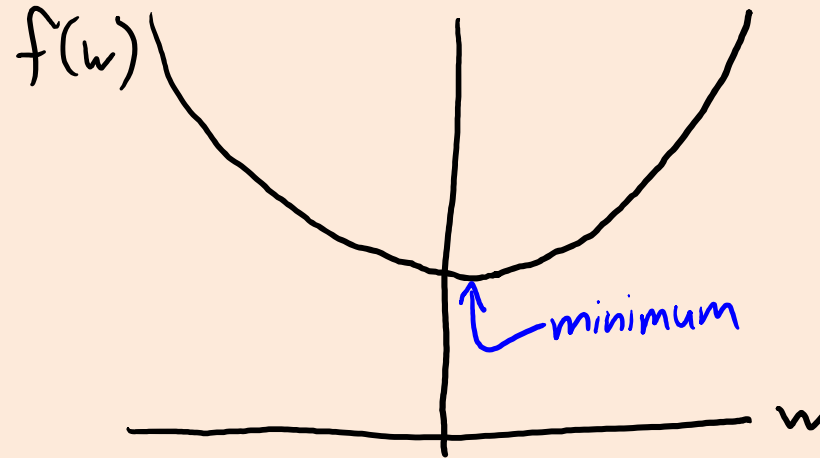
- L1-regularization focuses on decreasing all  $w_j$  until they are 0.

# Sparsity and Least Squares

- Consider 1D least squares objective:

$$f(w) = \frac{1}{2} \sum_{i=1}^n (w x_i - y_i)^2$$

- This is a convex 1D quadratic function of 'w' (i.e., a parabola):



- This variable does not look relevant (minimum is close to 0).
  - But for finite 'n' the minimum is unlikely to be exactly zero.

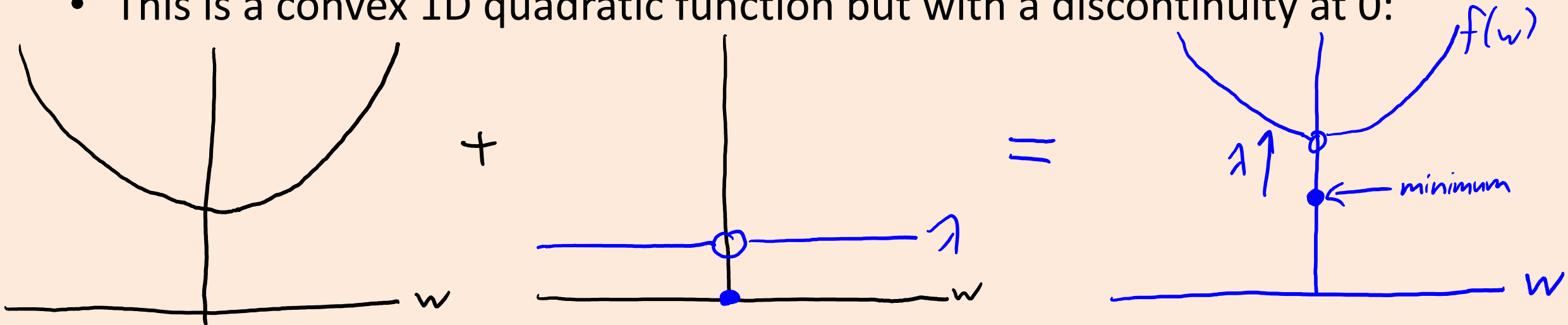
$f'(0) = 0$   
only happens  
if  $\sum_{i=1}^n y_i x_i = 0$ .  
(bonus)

# Sparsity and L0-Regularization

- Consider 1D **L0-regularized** least squares objective:

$$f(w) = \frac{1}{2} \sum_{i=1}^n (w x_i - y_i)^2 + \underbrace{\lambda \|w\|_0}_{\substack{\lambda \text{ if } w \neq 0 \\ 0 \text{ if } w = 0}}$$

- This is a convex 1D quadratic function but with a discontinuity at 0:



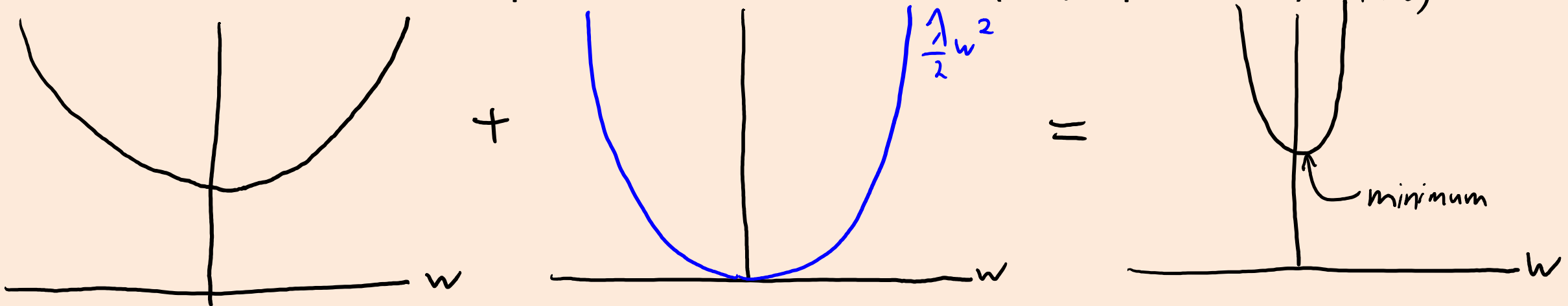
- L0-regularized minimum is often exactly at the 'discontinuity' at 0:
  - Sets the feature to exactly 0 (does feature selection), but is **non-convex**.

# Sparsity and L2-Regularization

- Consider 1D **L2-regularized** least squares objective:

$$f(w) = \frac{1}{2} \sum_{i=1}^n (w x_i - y_i)^2 + \frac{\lambda}{2} w^2$$

- This is a convex 1D quadratic function of 'w' (i.e., a parabola):  $f(w)$



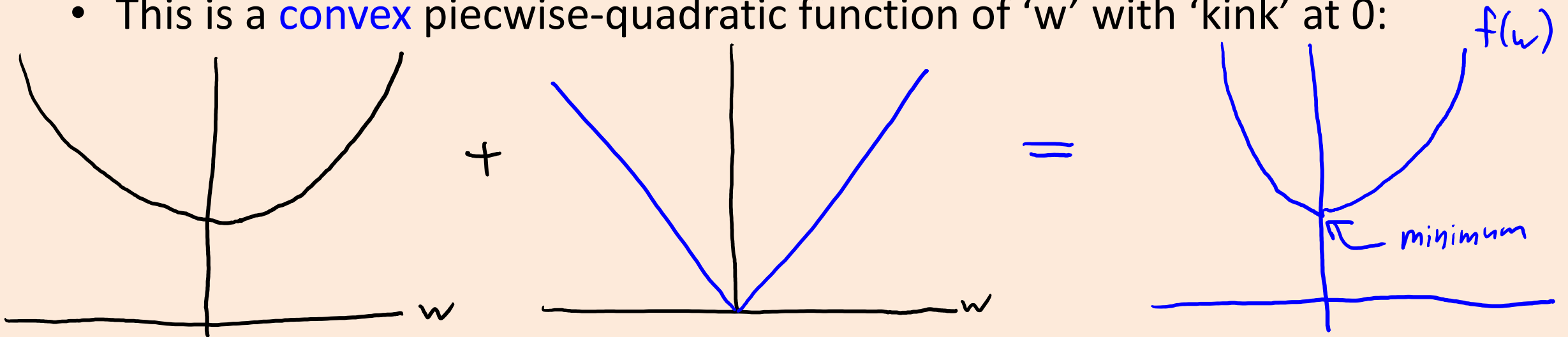
- L2-regularization moves it closer to zero, but not all the way to zero.
  - It **doesn't do feature selection** ("penalty goes to 0 as slope goes to 0"). →  $f'(0) = 0$   
only if  $\sum_{i=1}^n y_i x_i = 0$

# Sparsity and L1-Regularization

- Consider 1D **L1-regularized** least squares objective:

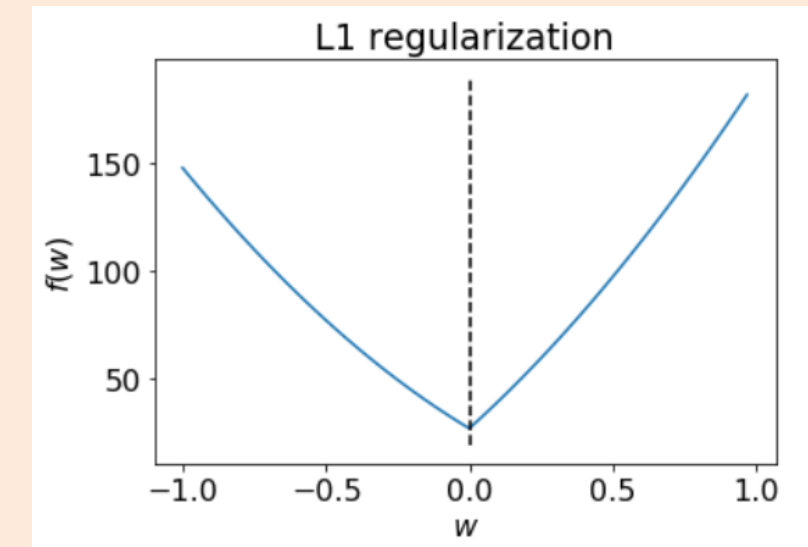
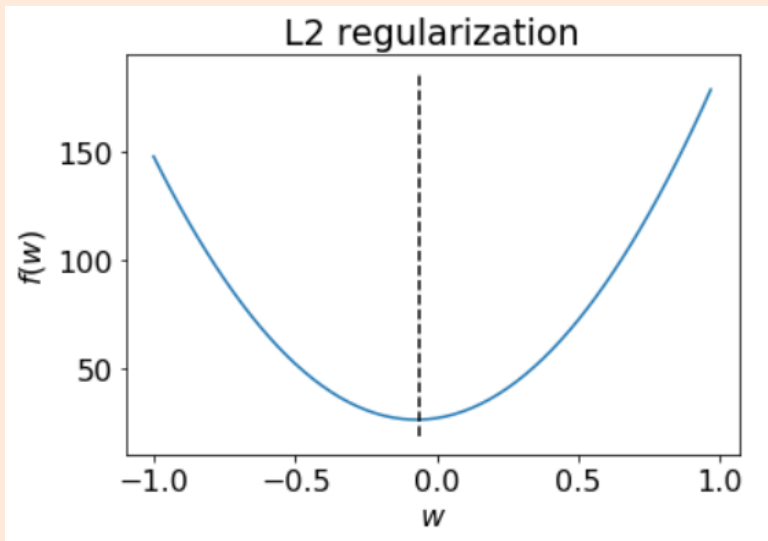
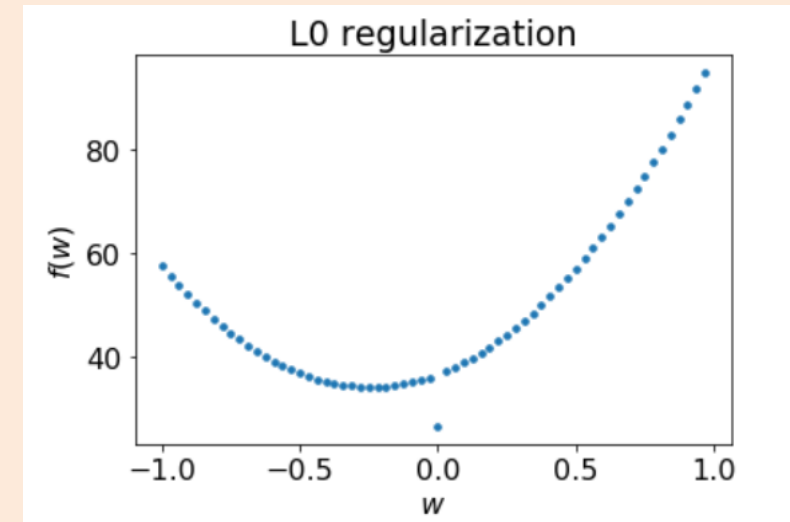
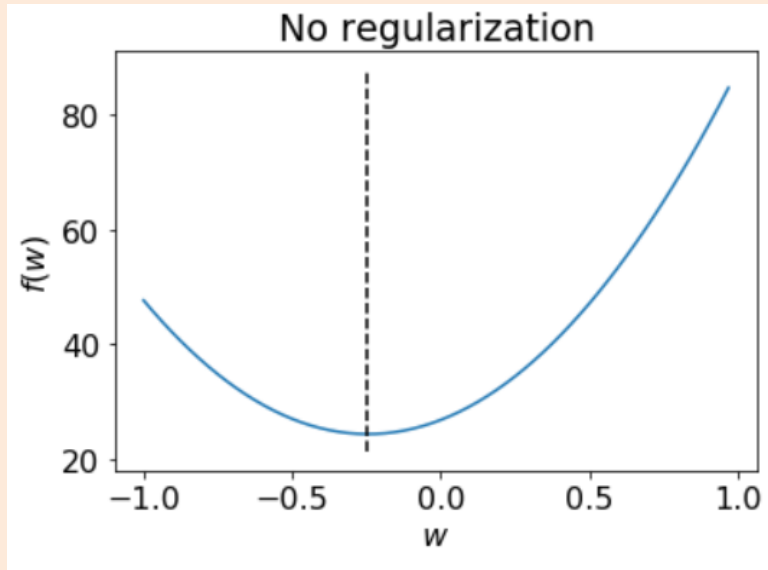
$$f(w) = \frac{1}{2} \sum_{i=1}^n (w x_i - y_i)^2 + \lambda |w|$$

- This is a **convex** piecewise-quadratic function of 'w' with 'kink' at 0:



- L1-regularization tends to **set variables to exactly 0** (feature selection).
    - Penalty on slope is  $\lambda$  even if you are close to zero.
    - Big  $\lambda$  selects few features, small  $\lambda$  allows many features.
- Happens when  $|\sum_{i=1}^n x_i y_i| \leq \lambda$  (bonus)*

# Sparsity and Regularization (with $d=1$ )



# Why doesn't L2-Regularization set variables to 0?

- Consider an L2-regularized least squares problem with 1 feature:

$$f(w) = \frac{1}{2} \sum_{i=1}^n (wx_i - y_i)^2 + \frac{\lambda}{2} w^2$$

- Let's solve for the optimal 'w':

$$f'(w) = \sum_{i=1}^n x_i (wx_i - y_i) + \lambda w$$

Set equal to 0:  $\sum_{i=1}^n x_i^2 w - \sum_{i=1}^n x_i y_i + \lambda w = 0$

re-arrange  $\rightarrow$

$$w \left( \underbrace{\sum_{i=1}^n x_i^2}_{\|x\|^2} + \lambda \right) = \underbrace{\sum_{i=1}^n x_i y_i}_{y^T x}$$

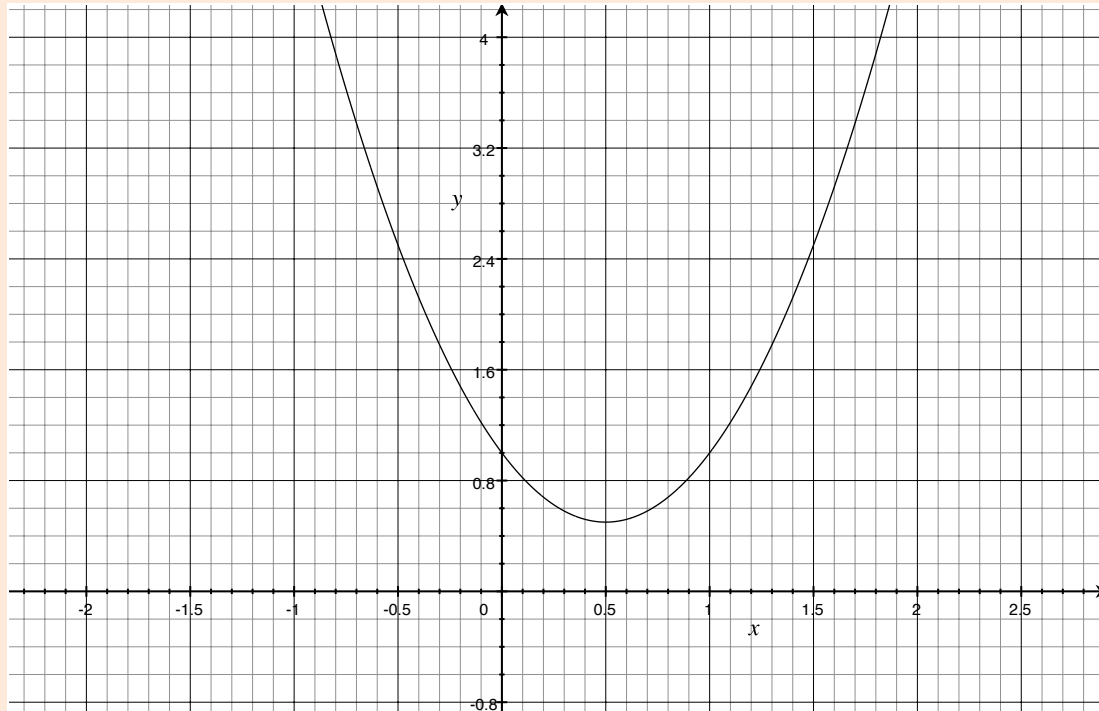
or  $w = \frac{y^T x}{\|x\|^2 + \lambda}$

- So as  $\lambda$  gets bigger, 'w' converges to 0.
- However, **for all finite  $\lambda$  'w' will be non-zero** unless  $y^T x = 0$  exactly.
  - But it's very **unlikely that  $y^T x$  will be exactly zero.**



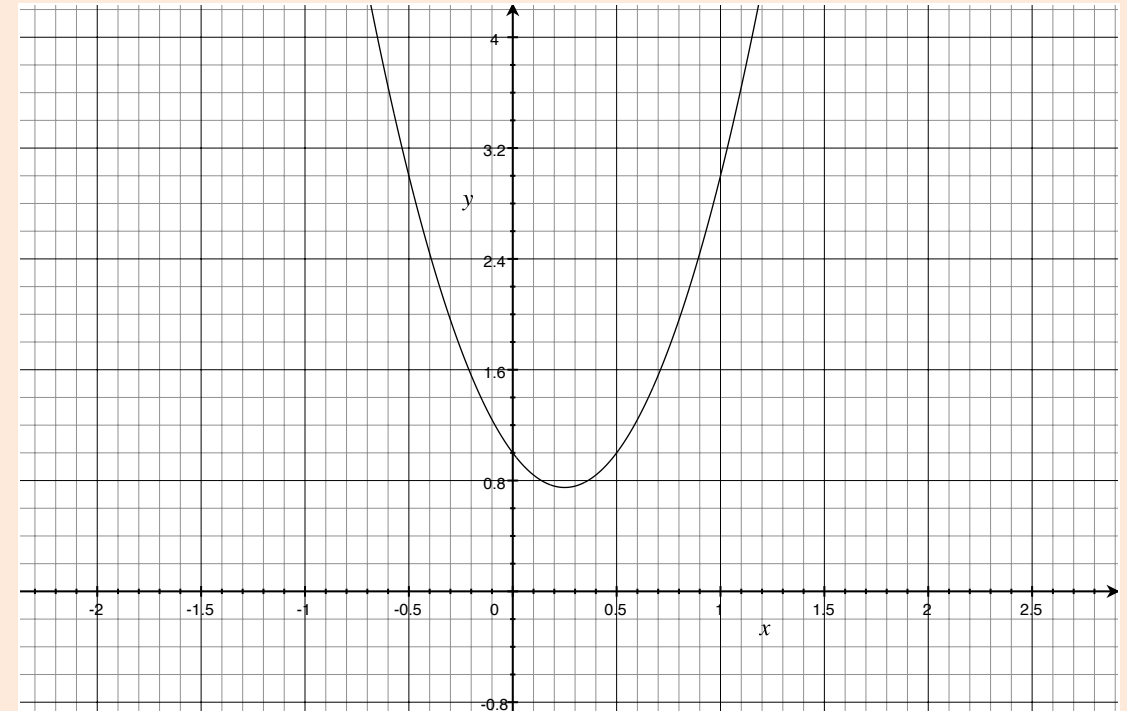
# Why doesn't L2-Regularization set variables to 0?

- Small  $\lambda$



- Solution further from zero

- Big  $\lambda$



Solution closer to zero  
(but not exactly 0)

# Why does L1-Regularization set things to 0?

- Consider an L1-regularized least squares problem with 1 feature:

$$f(w) = \frac{1}{2} \sum_{i=1}^n (w x_i - y_i)^2 + \lambda |w|$$

- If ( $w = 0$ ), then “left” limit and “right” limit are given by:

$$\begin{aligned} f^-(0) &= \sum_{i=1}^n x_i (0 x_i - y_i) - \lambda \\ &= \sum_{i=1}^n x_i y_i - \lambda \end{aligned}$$

$$\begin{aligned} f^+(0) &= \sum_{i=1}^n x_i (0 x_i - y_i) + \lambda \\ &= \sum_{i=1}^n x_i y_i + \lambda \end{aligned}$$

- So which direction should “gradient descent” go in?

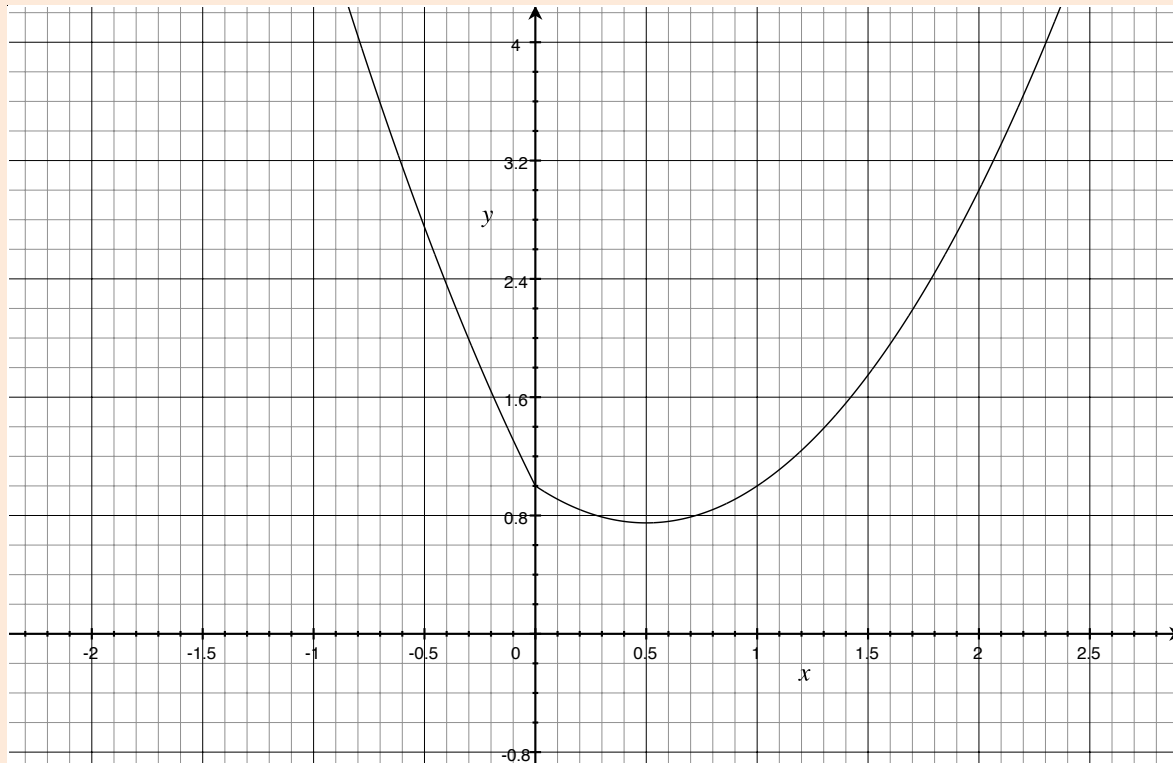
$$\begin{aligned} -f^-(0) &= -y^T x + \lambda \\ -f^+(0) &= -y^T x - \lambda \end{aligned} \quad \left. \begin{array}{l} \text{If these are positive } (-y^T x > \lambda), \\ \text{we can improve by increasing 'w'}. \end{array} \right\}$$

If these are negative ( $y^T x > \lambda$ ),  
we can improve by decreasing 'w'.

But if left and right “gradient descent” directions point in opposite directions ( $|y^T x| \leq \lambda$ ), minimum is 0.

# Why does L1-Regularization set things to 0?

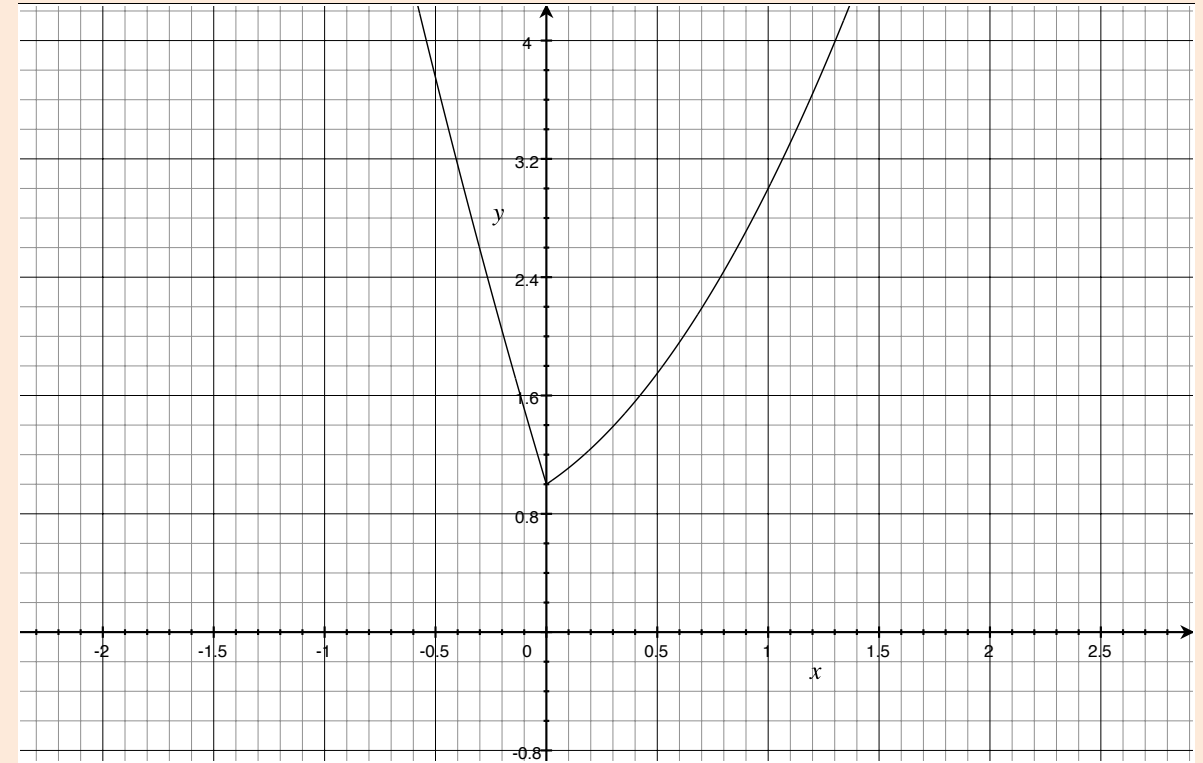
- Small  $\lambda$



- Solution nonzero

(minimum of left parabola is past origin, but right parabola is not)

- Big  $\lambda$



- Solution exactly zero

(minimum of both parabola are past the origin)

# L2-regularization vs. L1-regularization

- So with 1 feature:
  - L2-regularization only sets 'w' to 0 if  $y^T x = 0$ .
    - There is a **only a single possible  $y^T x$  value where the variable gets set to zero.**
    - And  **$\lambda$  has nothing to do with the sparsity.**
  - L1-regularization sets 'w' to 0 if  $|y^T x| \leq \lambda$ .
    - There is a **range of possible  $y^T x$  values where the variable gets set to zero.**
    - And **increasing  $\lambda$  increases the sparsity** since the range of  $y^T x$  grows.
- Note that it's **important that the function is non-differentiable:**
  - Differentiable regularizers penalizing size would need  $y^T x = 0$  for sparsity.

# L1-Loss vs. Huber Loss

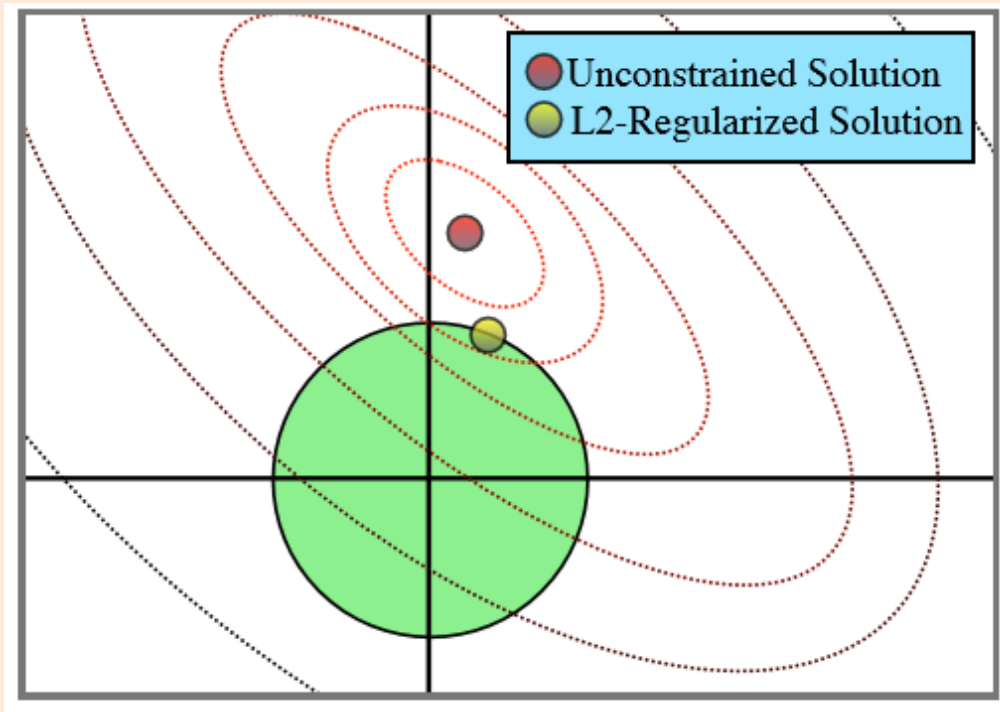
- The same reasoning tells us the difference between the L1 \*loss\* and the Huber loss. They are very similar in that they both grow linearly far away from 0. So both are both robust but...
  - With the L1 loss the model often passes exactly through some points.
  - With Huber the model doesn't necessarily pass through any points.
- Why? With L1-regularization we were causing the elements of 'w' to be exactly 0. Analogously, with the L1-loss we cause the elements of 'r' (the residual) to be exactly zero. But zero residual for an example means you pass through that example exactly.

# Non-Uniqueness of L1-Regularized Solution

- How can L1-regularized least squares solution not be unique?
  - Isn't it convex?
- Convexity implies that minimum value of  $f(w)$  is unique (if exists), but there may be **multiple 'w' values that achieve the minimum.**
- Consider L1-regularized least squares with  $d=2$ , where feature 2 is a copy of a feature 1. For a solution  $(w_1, w_2)$  we have:
$$\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} = w_1 x_{i1} + w_2 x_{i1} = (w_1 + w_2) x_{i1}$$
- So we can get the same squared error with different  $w_1$  and  $w_2$  values that have the same sum. Further, if neither  $w_1$  or  $w_2$  changes sign, then  $|w_1| + |w_2|$  will be the same so the new  $w_1$  and  $w_2$  will be a solution.

# L2-Regularization vs. L1-Regularization

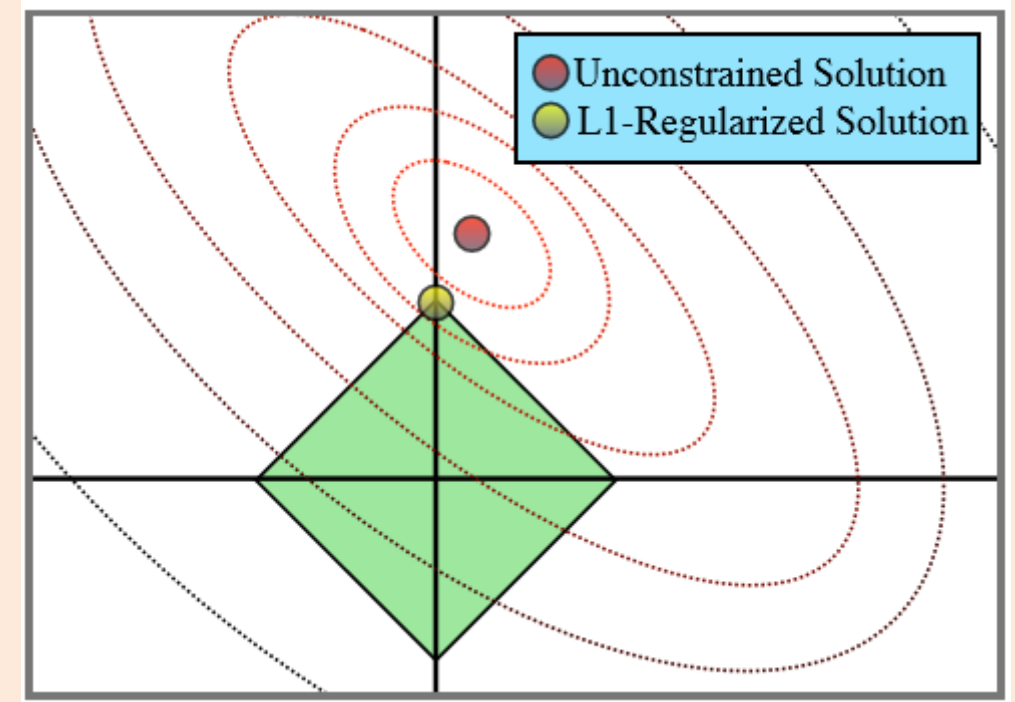
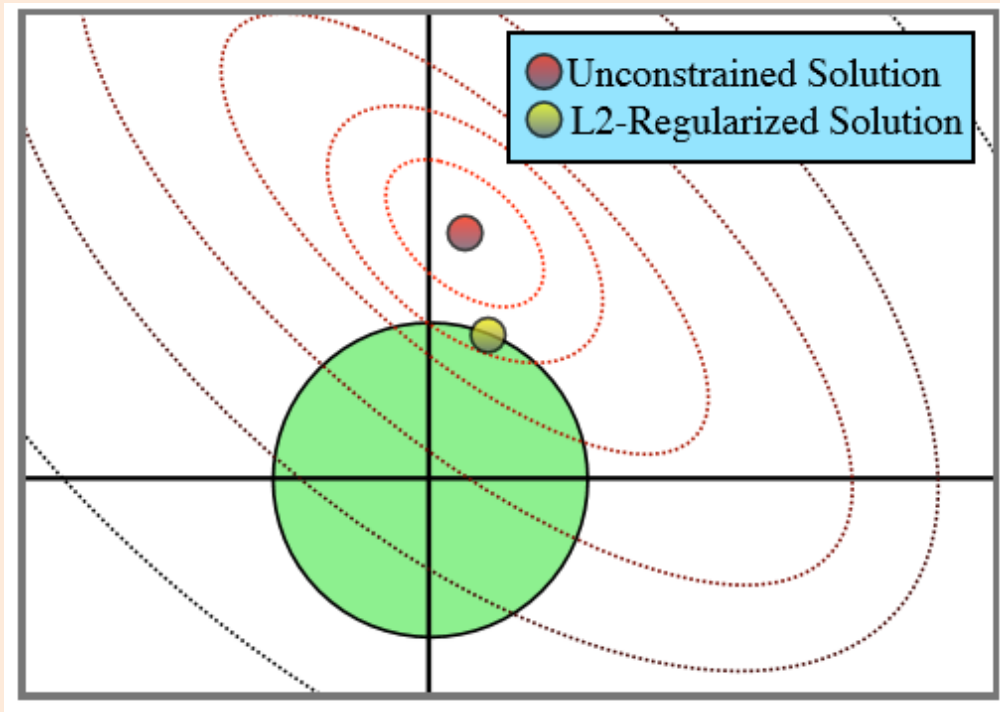
- L2-regularization conceptually restricts 'w' to a ball.



Minimizing  $\frac{1}{2} \|Xw - y\|^2 + \frac{\gamma}{2} \|w\|^2$   
is equivalent to minimizing  
 $\frac{1}{2} \|Xw - y\|^2$  subject to  
the constraint that  $\|w\| \leq \gamma$   
for some value ' $\gamma$ '

# L2-Regularization vs. L1-Regularization

- L2-regularization conceptually restricts ' $w$ ' to a ball.



- L1-regularization restricts to the L1 “ball”:
  - Solutions tend to be at corners where  $w_j$  are zero.